What is Bash?

The first lesson gives you a high-level view of what bash is, and a little on its history.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Other shells
- History of bash
- What You Learned
- What Next?
- Exercises

Bash is a *shell program*.

A *shell program* is typically an executable **binary** that takes commands that you type and (once you hit return), translates those commands into (ultimately) system calls to the Operating System API.

> Note: A binary is a file that contains the instructions for a program, ie it is a 'program' file, rather than a 'text' file, or an 'application' file (such as a Word document).

If you're not sure what an Operating System API is, then don't worry. You only need to know that a shell program is a program that allows you to tell the computer what to do. In that way, it's not much different to many other kinds of programming languages.

What makes bash different from some other languages is that it is a language designed to 'glue' together other programs.

How Important is this Lesson?

How important is this Lesson?

This lesson will delve into the history of bash. This information isn't essential for the topics covered in the rest of the course. However, it'll be useful for a thorough understanding of the subject.

Other shells

Bash is not the only kind of shell. Other shells include:

- `sh`
- `ash`
- `dash`
- `ksh`
- `tcsh`
- `zsh`
- `tclsh`

You might have heard about some of them or may have used them before. The other shells mentioned have different rules, conventions, logic, and histories meaning they can look similar, however, can differ in subtle ways.

Since other shells are also programs, they can be run from within one another!

In the following code sample you run `tcsh` from within the terminal. Note that you will get a different prompt after the `tcsh` command is run.

```
tcsh            # Enter the tcsh shell
echo $dirstack  # Output a variable in the tcsh shell
exit            # Quit the tcsh program, back to bash shell
echo $dirstack  # The variable does not exist in the bash shell
```

Type the above code into the terminal in this lesson.

Type the above commands in to the terminal below.

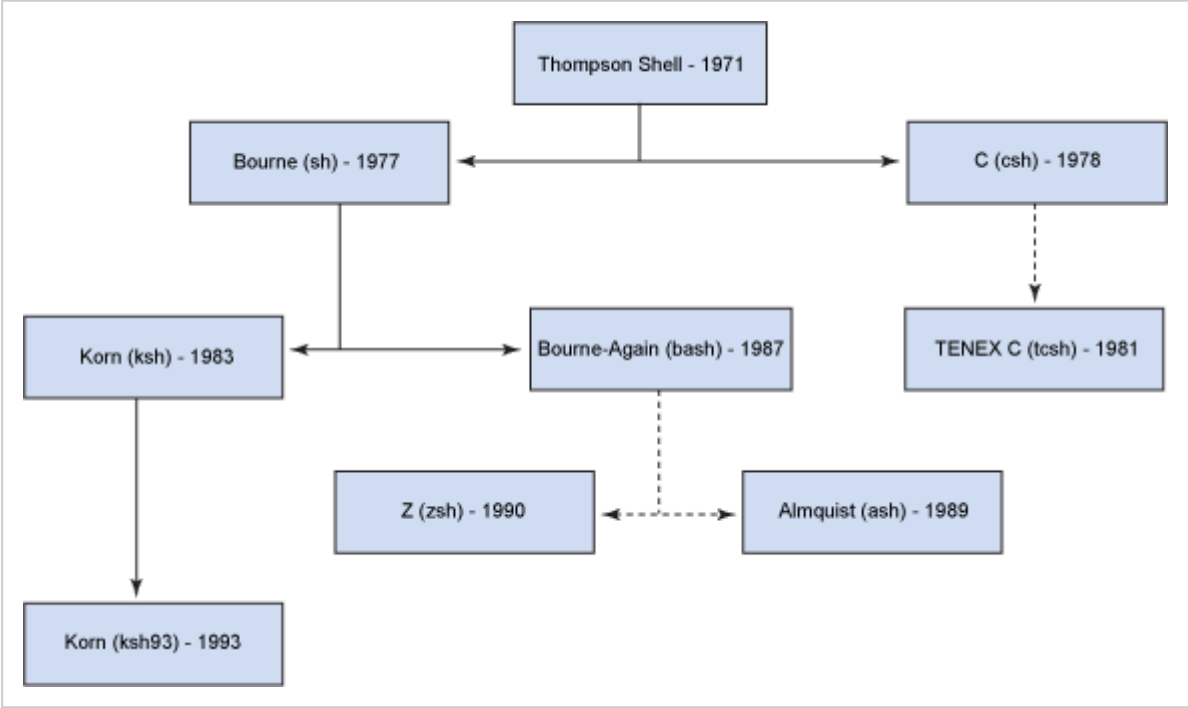
Terminal

> Note: Throughout this course it is assumed that you complete each lesson by typing in the commands in the provided terminal for that lesson in the order the commands are given.

The `dirstack` variable is set by tcsh and will output something meaningful. But once you have `exit`ed from the `tcsh` program, you are back in the bash shell, which does not have a `dirstack` variable by default.

History of bash

This diagram helps give a picture of the history of bash:



A family tree of shells

Bash is called the 'Bourne Again SHell'. It is a descendant of the 'Thompson Shell' and then the Bourne 'sh' shell. Bash has other 'siblings' (eg, `ksh`), 'cousins' (eg, `tcsh`), and 'children', (eg, `zsh`).

The details aren't necessary to know at this point, but it is important to know that different shells exist that can be related and somewhat compatible.

Bash is the most widely seen, used and available shell. However, it is still not

unheard of to end up on servers that do not have bash!

What is Bash? Quiz

1 What is bash's historical 'parent' shell?

COMPLETED 0%

1 of 3

What You Learned

- What a *shell* is
- How to start up a different *shell*
- The family tree of shells

What Next?

Next you look at two thorny but ever-present subjects in bash: **globbing** and **quoting**.

Exercises

1) Run `sh` from a bash command line. What happens?

2) What commands can you find that work in `bash`, but do not work in `sh`?

Globbing and Quoting

This lesson will introduce you to the joy of quoting in bash. You will learn what globbing is and what its special characters are. It'll also explain the difference between single and double quotes, and you'll learn the difference between globs and regular expressions.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Globbing
- Quoting
- Other glob characters
- Dotfiles
- Differences with Regular Expressions
- What You Learned
- What Next?
- Exercises

If you've come across it before, you may have wondered what the `*` in bash commands really means, and how it is different from *regular expressions*.

> Note: Do not panic if you don't know what regular expressions are. Regular expressions are patterns used to search for matching strings. Globs look similar and perform a similar function, but are not the same. That's the key point to understand about globs vs regular expressions.

How Important is this Lesson?

Globbing and quoting are essential topics when using bash. It's rare to come across a set of commands or a script that doesn't depend on knowledge of them.

Globbing

Type these commands into the terminal:

```
touch file1 file2 file3
ls *
echo *
```

Type the above code into the terminal in this lesson.

- **Line 1** creates three files (`file1`, `file2`, `file3`)
- **Line 2** runs the `ls` command, asking to list the files matching `*`
- **Line 3** runs the `echo` command using `*` as the argument to `echo`

Type all the commands in this lesson out in the terminal below.

Terminal

> Note: continue to use this terminal during this lesson. Throughout this course it is assumed that you complete each lesson by typing in the commands in the provided terminal for that lesson in the order the commands are given.

You should be able to see the three files listed when both, **line 4** and **line 5**, are executed.

The shell took the `*` character and converted it to match all the files in the current working directory. In other words, it has converted the `*` character into the string `file1 file2 file3` and then processed the resulting command.

Quoting

What do you think will be output if we run these commands?

Think about it first, make a prediction, and then type it out!

```
ls '*'
ls "*"
echo '*'
echo "*"
```

Type the above code into the terminal in this lesson.

- **Line 1** lists files matching the `*` character in single quotes
- **Line 2** lists files matching the `*` character in double quotes
- **Line 3** echoes the `*` character in single quotes
- **Line 4** echoes the `*` character in double quotes

You will see that the first two commands result in an error, and the second two just show the raw `*` on the terminal. In other words, both types of quote do nothing with the `*` character.

This is difficult to predict even if you are very experienced in bash!

Was the output what you expected? Can you explain it? Ironically it may be harder to explain if you have experience of quoting variables in bash!

Quoting in bash is a very tricky topic. The point to be noted is that quoting globs removes their effect. But in other contexts single and double quotes have different meanings. You will see example of this in the next lesson on variables.

Quoting changes the way bash can read the line, making it decide whether to take these characters and transform them into something else, or just leave them be. We will see more examples of this throughout the course, and specifically in the next lesson

What you should take from this is that "quoting in bash is tricky" and be prepared for some head-scratching later!

Other glob characters

`*` is not the only globbing primitive. Other globbing primitives are:

- `?` - matches any single character
- `[abd]` - matches any character from a, b or d
- `[a-d]` - matches any character from a, b, c or d

Try running these commands and see if the output is what you expect:

```
ls *1
ls file[a-z]
ls file[0-9]
```

Type the above code into the terminal in this lesson.

- **Line 1** lists all the files that end in '1'
- **Line 2** lists all files that start with 'file' and end with a character from a to z
- **Line 3** lists all files that start with 'file' and end with a character from 0 to 9

Dotfiles

Dotfiles are like normal files, except their name begins with a dot. Create some with `touch` and `mkdir`:

```
touch .adotfile
mkdir .adotfolder
touch .adotfolder/file1 .adotfolder/.adotfile
```

Type the above code into the terminal in this lesson.

You've now created some dotfiles and folders. If you run `ls`:

```
ls
```

Type the above code into the terminal in this lesson.

those files don't show up. So these files are hidden from us in normal view. What if we try to use a * as a glob?

```
ls *
```

Type the above code into the terminal in this lesson.

Same result. Those files are hidden. While this may seem (and sometimes is) annoying, having files that don't match even a `*` glob is very useful.

Frequently you want to have a file that sits alongside other files but that is generally ignored.

For example, you might write some code that reformats a set of text files in a folder, but you don't want to reformat a dotfile that contains information about what's in those text files.

Unfortunately, it can be annoying when you really do want to see *all* the files in a folder. To achieve this, type:

```
echo .*
```

Type the above code into the terminal in this lesson.

The leading dot tells bash that you really do want all the files to be matched.

If you do the same with `ls`:

```
ls .*
```

Type the above code into the terminal in this lesson.

you get a slightly more complicated output, as `ls` returns richer output depending on whether the item is a file or a folder. If it's a folder, it shows every file within that folder under a separate heading.

You'll also get a couple of extra 'special' folders returned that you may not have been aware of before.

The single dot folder (`.`) is a special folder that represents the folder that you are in. For example, if you type:

```
cd .
```

Type the above code into the terminal in this lesson.

You will go nowhere! You've changed directory to the same folder that you are in.

The double dot folder (`..`) is a special folder that represents the folder that represents the parent folder of the one you are in.

What do you think happens at the root folder (`/`)? Have a look and find out.

Differences with Regular Expressions

While globs look similar to regular expressions (regexes), they are used in different contexts and are separate things.

The * character in this command has a different significance depending on whether it is being treated as a glob or a regular expression.

```
rename -n 's/(.*)/new$1/' *
```

Type the above code into the terminal in this lesson.

You should see output like this:

```
rename(file1, newfile1)
rename(file2, newfile2)
rename(file3, newfile3)
```

- **Line 1** contains the command that renames all filenames to prepend 'new' in front. The `-n` flag tells rename to just print out the files that would be changed, and not actually carry out the renaming

The first `*` character is treated as regular expressions, because it is not interpreted by the shell, but rather by the `rename` command. The reason it is not interpreted by the shell is because it is enclosed in single quotes. The last `*` is treated as a glob by the shell, and expands to all the files in the local directory.

Again, the key takeaway here is that context is key.

Note that `.` has no meaning as a glob, and that some shells offer more powerful extended globbing capabilities. Bash is one of the shells that offers extended globbing, which we do not cover here, as it would potentially confuse the reader further. Just be aware that more sophisticated globbing is possible.

Globbing and Quoting Quiz

1

What will this command output in an empty folder?

`ls *`

COMPLETED 0%

1 of 2

What You Learned

- What a *glob* is
- What a *dotfile* is
- *Globs* and *regexes* are different
- Single and double quotes around *globs* can be significant!

What Next?

Next up is another fundamental topic: **variables**.

Exercises

1) Create a folder containing files with very similar names and use globs to list one and not the other.

2) Research regular expressions online.

3) Research the program `grep`. If you already know it, read the `grep` `man`

3) Research the program `grep`. If you already know it, read the `grep` `man` page. (Type `man grep`).

Variables

In this lesson, you'll learn about variables in bash and some of their subtleties. It will cover basic variables and how they're quoted, the 'env' and 'export' commands, and simple and associative arrays. By the end, you will have a good overview of how variables work in bash and some of their pitfalls.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Basic Variables
- Variables and Quoting
- Shell Variables
- The `export` Command
- Outputting Exported and Shell Variables
- Arrays
- Associative Arrays
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Variables are fundamental to understanding bash commands, as much as they are in any programming language.

Basic Variables

Start by creating a variable and `echo`ing it.

```
MYSTRING=astring
echo $MYSTRING
```

Type the above code into the terminal in this lesson.

Terminal

> Note: continue to use this terminal during this lesson. Throughout this course it is assumed that you complete each lesson by typing in the commands in the provided terminal for that lesson in the order the commands are given.

Simple enough: you create a variable by

- Stating its name
- Immediately adding an equals sign
- Immediately stating the value

Variables don't need to be capitalized, but they generally are by convention.

To get the value out of the variable, you have to use the dollar sign to tell bash that you want the variable dereferenced.

Variables and Quoting

Things get more interesting when you start quoting.

Quoting can be used to group different *words* into a single variable value:

```
MYSENTENCE=A sentence     # This command will not work
MYSENTENCE="A sentence"   # The quotes group the words together
echo $MYSENTENCE
```

Type the above code into the terminal in this lesson.

Since (by default) the shell reads each *word* separated by a space, it thinks the word `sentence` is not related to the variable assigment, and treats it as a program. To store a sentence with space in it inside a variable, you can enclose it in the double quotes, as shown above.

Things get even more interesting when we embed other variables in the quoted string:

```
MYSENTENCE="A sentence with $MYSTRING in it"
echo $MYSENTENCE                              # Outputs the sentence with the variable deref
MYSENTENCE='A sentence with $MYSTRING in it'
```

```
echo $MYSENTENCE                                # Outputs the sentence with the variable not c
```

Type the above code into the terminal in this lesson.

If you were expecting similar behaviour to the previous lesson you may have gotten a surprise!

This illustrated an important point if you're reading shell scripts: the bash shell translates the variable into its value if it's in double quotes, but does not if it's in single quotes.

Remember from the previous lesson that this is not true when globbing!

Try out the code below in the terminal and see the output. Like always, make sure you think about the output you expect before you see it:

```
MYGLOB=*        # No quotes around the glob
echo $MYGLOB    # Glob is interpreted
MYGLOB="*"      # Double quotes around the glob
echo "$MYGLOB"  # Glob is not interpreted
MYGLOB='*'      # Single quotes around the glob
echo "$MYGLOB"  # Glob is not interpreted
echo '$MYGLOB'  # Variable is not interpreted
echo $MYGLOB    # Glob is interpreted
```

Type the above code into the terminal in this lesson.

Globs are not expanded when in either single or double quotes. Confusing isn't it?

Shell Variables

Some variables are special, and set up when bash starts:

```
echo $PPID
PPID=nonsense
echo $PPID
```

Type the above code into the terminal in this lesson.

- **Line 1** - `PPID` is a special variable set by the bash shell. It contains the bash's parent process id
- **Line 2** - Try and set the `PPID` variable to something else
- **Line 3** - Output `PPID` again

Like 3 Output `PPID` again

Can you work out what happened there?

You couldn't set the variable, because this is a `readonly` variable.

If you want to make a variable read-only, put `readonly` in front of it, like this:

```
readonly MYVAR=astring
MYVAR=anotherstring
```

Type the above code into the terminal in this lesson.

The `export` Command

Type in these commands, and try to predict what will happen:

```
MYSTRING=astring                # Set the MYSTRING variable
bash                            # Enter a fresh bash shell
echo $MYSTRING                  # Has the string made it to the new bash shell?
exit                            # Quit the bash shell
echo $MYSTRING                  # Is the string still there?
unset MYSTRING                  # Unset the string
echo $MYSTRING                  # Check it's not there
export MYSTRING=anotherstring   # Do the same, but export it this time
bash                            # Enter a fresh bash shell
echo $MYSTRING                  # Has it made it this time?
exit
```

Type the above code into the terminal in this lesson.

Based on this, what do you think `export` does?

You've already seen that a variable set in a bash terminal can be referenced later by using the dollar sign.

But what happens when you set a variable, and then start up another process?

In this case, you set a variable (`MYSTRING`) to the value `astring`, and then start up a new bash shell process. Within that bash shell process, `MYSTRING` does not exist, so an error is thrown. In other words, the variable was not inherited by the bash process you just started.

After exiting that bash session, and unsetting the `MYSTRING` variable to ensure it's gone, you set it again, but this time `export` the variable, so that any processes started by the running shell will have it in their environment. You show this by starting up another bash shell, and it `echoes` the new value

`anotherstring` to the terminal.

It's not just shells that have environment variables! All processes have environment variables.

Outputting Exported and Shell Variables

Wherever you are, you can see the exported variables that are set by running `env`:

```
env
```

Type the above code into the terminal in this lesson.

The output of `env` will likely be different wherever you run it.

That isn't all the variables that are set in your shell, though. It's just the *environment* variables that are exported to processes that you start in the shell.

If you want to see *all* the variables that are available to you in your shell, type:

```
compgen -v
```

Type the above code into the terminal in this lesson.

`compgen` is a command that generates list of possible 'word completions' in bash when you hit tab repeatedly. The `-v` flag shows all the variables that could be completed from where you type. Hence, it lists all variables, exported and local to the shell, in the environment where you are.

Arrays

Also worth mentioning here are *arrays*. One such built-in, read only array is `BASH_VERSINFO`. As in other languages, arrays in bash are zero-indexed.

Type out the following commands, which illustrate how to reference the version information's major number:

```
echo $BASH_VERSINFO
echo $BASH_VERSINFO[0]   # Does not work
echo ${BASH_VERSINFO[0]} # Will work
```

```
echo ${BASH_VERSINFO}
```

Type the above code into the terminal in this lesson.

Arrays can be tricky to deal with, and bash doesn't give you much help!

The first thing to notice is that the array will output the item at the index `0` if no index is given.

The second thing to notice is that simply adding `[0]` to a normal variable reference does not work. Bash treats the square bracket as a character not associated with the variable and appends it to the end of the array.

You have to tell bash to treat the whole string `BASH_VERSION[0]` as the variable to be dereferenced. You do this by using the curly braces.

These curly braces can be used on simple variables too:

```
echo $BASH_VERSION_and_some_string     # Won't find the variable
echo ${BASH_VERSION}_and_some_string   # The curly quotes tell bash where the variable is
```

Type the above code into the terminal in this lesson.

In fact, 'simple variables' can be treated as arrays with one element!

```
echo ${BASH_VERSION[0]}
```

Type the above code into the terminal in this lesson.

So all bash variables are 'really' arrays!

Bash has 6 items (0-5) in its `BASH_VERSINFO` array:

```
  echo ${BASH_VERSINFO[1]}
  echo ${BASH_VERSINFO[2]}
  echo ${BASH_VERSINFO[3]}
  echo ${BASH_VERSINFO[4]}
  echo ${BASH_VERSINFO[5]}
  echo ${BASH_VERSINFO[6]}
```

Type the above code into the terminal in this lesson.

As ever with variables, if the item does not exist then the output will be an empty line.

Associative Arrays

Bash also supports *associative arrays*.

With associative arrays, you use a string instead of a number to reference the value:

```
declare -A MYAA=([one]=1 [two]=2 [three]=3)  # declare the associative array
MYAA[one]="1"                                # set the 'one' array element
MYAA[two]="2"
echo $MYAA                                   # outputs nothing
echo ${MYAA[one]}                            # outputs '1'
WANT=two
echo ${MYAA[$WANT]}                          # outputs '2'
```

Type the above code into the terminal in this lesson.

Variables Quiz

1

What will this output?

```
echo '$HOME'
```

COMPLETED 0%

1 of 2

As well as not being compatible with versions less than 4, associative arrays

are quite fiddly to create and use, hence are not seen very often.

What You Learned

- Basic variable usage in bash
- Variables and quoting
- Variables set up by bash
- `env` and `export`
- Bash arrays

What Next?

Next you will learn about another core language feature implemented in bash: **functions**.

Exercises

1) Take the output of `env` in your terminal and work out why each item is there and what it might be used for. You may want to use `man bash`, or google to figure it out. You could try resetting it to see what happens.

2) Find out what the items in `BASH_VERSINFO` mean.

Functions

This lesson discusses the four types of a ‘command’ that exist in bash: functions, aliases, programs, and built-ins.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Basic Functions
- Arguments
- Variable Scope
- Functions, Builtins, Aliases and Programs
- Builtins
- Functions
- Programs
- Aliases
- The ‘type’ Builtin
- What You Learned
- What Next?
- Exercises

From one angle, bash can be viewed as a programming language, albeit a quite slow and primitive one.

One of the language features it has is the capability to create and call *functions*.

This leads us onto the topic of what a ‘command’ can be in bash, and we cover all four of them: *functions*, *aliases*, *programs*, and *builtins*.

By the end you will have a more nuanced understanding of commands in bash.

How Important is this Lesson?

It's possible to get by without knowing much about functions in bash, but any serious bash user will know what they do and how they work.

Basic Functions

Start by creating a simple function:

```
function myfunc {
    echo Hello World
}
myfunc
```

Type the above code into the terminal in this lesson.

Terminal

> Note: continue to use this terminal during this lesson. Throughout this course it is assumed that you complete each lesson by typing in the commands in the provided terminal for that lesson in the order the commands are given.

By declaring a function, and placing the block of code that needs to run inside the curly braces, you can call that function on the command line as though it were a program.

Arguments

Unlike other languages, in bash there is no checking of functions' arguments.

Predict the output of this, and then run it:

```
function myfunc {
    echo $1
    echo $2
}
myfunc "Hello World"
myfunc Hello World
```

Type the above code into the terminal in this lesson.

Can you explain the output? If not, you may want to re-read the previous

Can you explain the output? If not, you may want to re-read the previous lessons!

Arguments to functions are numbered, from **1 to n**. It's up to the function to manage these arguments.

In the above example, in **line 5**, the quotes turn the `Hello World` string into a single argument, which is echoed on a single line. It is then followed by an empty line, because there was no second argument provided. In **line 6**, the `Hello World` string is not enclosed in quotes, so is treated as two separate variables, which are `echo`ed on two separate lines as two separate arguments.

This illustrates a very important point about shells, which is that they separate items by space by default. This will become very important when we discuss the `IFS` in a later lesson

Variable Scope

Variables can have scope in bash. This is particularly useful in functions, where you don't want your variables to be accessible from outside the function.

These commands illustrate this:

```
function myfunc {
    echo $myvar
}
myfunc
myvar="Hi from outside the function"
myfunc
```

Type the above code into the terminal in this lesson.

Bash functions have no special scope. Variables outside are visible to it.

There is, however, the capability within bash to declare a variable as local:

```
function myfunc {
    local myvar="Hi from inside the function"   # Creates a variable local to the function
    echo $myvar
}
myfunc
echo $myvar
local myvar="Will this work?"                   # Will not work
```

Type the above code into the terminal in this lesson.

The variable declared with `local` can only be viewed and accessed within the function, hence, it doesn't interfere with the code outside the function. As seen from the example above, it can't be declared outside a function.

The `local` above is an example of a bash 'builtin'. Now is a good time to talk about the different types of commands.

Functions, Builtins, Aliases and Programs

There are at least four ways to call commands in bash:

- Builtins
- Functions
- Programs
- Aliases

Let's take a look at each type of command one by one.

Builtins

Builtins are commands that come 'built in' to the bash shell program. Normally you can't easily tell the difference between a builtin, a program or a function, but after reading this you will be able to.

The most commonly-used builtin is `cd`. There is also one called `builtin`!

```
builtin cd /tmp        # Call cd as normal with the builtin builtin
cd -
builtin grep           # Is 'grep' a builtin?
builtin notaprogram    # What happens if the command doesn't exist at all
```

Type the above code into the terminal in this lesson.

As you've probably guessed by typing the above commands in the terminal, the `builtin` builtin calls the builtin program (this time `cd`), and throws an error if no such builtin exists.

In case you didn't know, `cd -` returns you to the previous directory you were in.

Functions

Functions we have covered above, but what happens if we write a function that clashes with a builtin?

What if you create a function called `cd`?

```
function cd() {
    echo 'No!'
}
cd /tmp
builtin cd /tmp
cd -
unset -f cd
cd /tmp
cd -
```

Type the above code into the terminal in this lesson.

- In **lines 1-3** you created a function called cd that just outputs `No!`
- In **line 4** you then tried to move to another folder, and saw that the function handled the `cd` command over the built-in `cd` command
- In **line 5** you explicitly called the `cd` built-in using the `builtin` command, which ensures that the `builtin` is called over a function or a program
- After returning to the previous folder with `cd -` in **line 6** you `unset` the function with the `unset -f` built-in command in **line 7**, after which `cd` can be used as normal in **lines 8 and 9**

Now type this in:

```
function cd() {
    echo 'Function cd'
}
declare -f              # Show functions in the environment
declare -F              # Show just the function names
unset -f cd
```

Type the above code into the terminal in this lesson.

If you want to know what functions are set in your environment, you run `declare -f`. This will output the functions and their bodies. If you just want the names, use the `-F` flag.

Programs

Programs are executable files. Commonly-used examples of these are programs such as `grep`, `sed`, `vi`, and so on.

How do you tell whether a command is a `builtin` or a separate binary?

First, see whether it's a `builtin` by running `builtin <command>` as you did before. Then you can also run the `which` command to determine where the file is stored on your filesystem.

```
which grep              # Where is the grep program?
which cd
which builtin
which doesnotexist
```

Type the above code into the terminal in this lesson.

Is `which` a builtin or a program?

Show Hint

Aliases

Finally there are **aliases**. Aliases are strings that the shell takes and translates to whatever that string is aliased to.

Try this and explain to yourself what is going on as you go:

```
alias cd=doesnotexist
alias
cd
unalias cd
cd /tmp
cd -
alias
```

Type the above code into the terminal in this lesson.

- In **line 1** you alias the `cd` command to the `doesnotexist` command (which does not exist)
- **Line 2** shows the aliases available

- **line 3** shows that the alias has 'taken over' the cd command
- `unalias` in **line 4** removes the `alias` you created, so that
- **lines 5-6** `cd` can be used as normal again
- Running `alias` again in **line 7** shows that the `alias` for `cd` is no longer there

The 'type' Builtin

There is also a builtin called `type` that tells you how a command would be interpreted by the shell:

```
type ls       # What kind of command is 'ls'?
type pwd
type myfunc
```

Type the above code into the terminal in this lesson.

Functions Quiz

1 Choose the true statements

COMPLETED 0%

1 of 2

What You Learned

- Basic function creation in bash
- Functions and variable scope
- Differences between functions, builtins, aliases and programs

What Next?

Next you will learn about **pipes and redirects** in bash. Once learned, you will have all you need to get to writing shell scripts in earnest.

Exercises

1) Run `typeset -f`. Find out how this relates to `declare -f` by looking at the bash man page (`man bash`).

2) alias `alias`, override `cd`. Try and break things. Have fun. If you get stuck, close down your terminal, or exit your bash shell (if you haven't overridden exit!).

Pipes and Redirects

In this lesson you'll look at basic redirection and pipes. You'll also learn about file descriptors, 'standard out' and 'standard error' and 'special' files like '/dev/null'.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Basic Redirects
- Basic Pipes
- Standard Output vs Standard Error
- Differences Between Pipes and Redirects
- What You Learned
- What Next?
- Exercises

Pipes and redirects are used very frequently in bash and by all levels of user.

This can cause a problem. They are used so often by all users of bash that many don't understand their subtleties, how they work, or their full power.

How Important is this Lesson?

This lesson is essential.

Basic Redirects

Start off by creating a file:

```
echo "contents of file1" > file1
```

Type the above code into the terminal in this lesson.

Terminal

> Note: continue to use this terminal during this lesson. Throughout this course it is assumed that you complete each lesson by typing in the commands in the provided terminal for that lesson in the order the commands are given.

The `>` character is the *redirect* operator. This takes the output from the preceding command that you'd normally see in the terminal and sends it to a file that you give it. Here it creates a file called `file1` and puts the `echo`ed string into it. Try `cat`ing the file if you want to check by running:

```
cat file1
```

There's a subtlety here which we'll get to: sometimes not all the output you see in the terminal would get redirected by this, but don't worry about this yet.

Basic Pipes

Now type this in:

```
cat file1 | grep -c file
```

Type the above code into the terminal in this lesson.

> Note: If you don't know what `grep` is, you will need to learn. This is a good place to start: https://en.wikipedia.org/wiki/Grep

Normally you'd run a `grep` with the filename as the last argument, but instead here we **pipe** the contents of `file1` into the `grep` command by using the 'pipe' operator: `|`. The resulting output of `1` indicates the number lines that matched the word `file` in the the file called `file1`.

A *pipe* takes the standard output of one command and passes it as the input to another. What, then is **standard output**, really? You will find out soon!

```
cat file2
```

Type the above code into the terminal in this lesson.

You should have seen an error, because the file did not exist.

Now run this, and try and guess the result before you run it:

```
cat file2 | grep -c file
```

Type the above code into the terminal in this lesson.

You should have seen two lines. The first will be an error

```
cat: file2: No such file or directory
```

and the second line a `0`.

We will explain further below why a `0` is shown, even though a line was outputted that had the word `file` in it.

> Note: See Exercise 3 below for an often-useful operator that changes this behaviour.

Standard Output vs Standard Error

In addition to **standard output**, there is also a **standard error** channel. When you pass a non-existent file to the `cat` command, it throws an error message out to the terminal, as you would have seen when you attempted the code above. Although the message looks the same as the contents of a file, it is sent to a different output channel. In this case it's *standard error* rather than *standard output*.

As a result, it is NOT passed through the pipe to the `grep` command, and `grep` counts zero matches in its output.

To the viewer of the terminal, there is no difference, but to bash there is all the difference in the world!

There is a simpler way to refer to these channels. A number is assigned to each of them by the operating system.

These are the numbered *file descriptors*, and the first three are assigned to the numbers `0`, `1` and `2`.

- `0` is *standard input*
- `1` is *standard output*
- `2` is *standard error*

When you redirect 'standard output' to a file, you use the redirection operator `>`. Implicitly, you are using the `1` file descriptor.

Type this to see an example of redirecting `2`, which is 'standard error'.

```
command_does_not_exist
command_does_not_exist 2> /dev/null
```

Type the above code into the terminal in this lesson.

- **Line 1** above will show an error as you might expect, but **line 2** does not
- In **line 2**, the file descriptor `2` (standard error) is directed to a file called `/dev/null`

The file `/dev/null` is a special file created by Linux (and UNIX) kernels. It is effectively a black hole into which data can be pumped: anything written to it will be absorbed and ignored.

Another commonly seen redirection operator is `2>&1`:

```
command_does_not_exist 2>&1
```

Type the above code into the terminal in this lesson.

What this does is tell the shell to send the output on standard error (`2`) to whatever endpoint standard output is pointed to at that point in the command.

Since standard output is pointed at the terminal at that time, standard error is also pointed at the terminal. From your point of view you see no difference, since both standard output and standard error are pointed at the terminal anyway.

But when we try and redirect to standard error or standard output to files things get interesting, as you can change where they go. You saw this above when we redirected standard error to `/dev/null`.

Now type these in and try and figure out why they produce different output:

```
command_does_not_exist 2>&1 > outfile
cat outfile
command_does_not_exist > outfile 2>&1
cat outfile
```

Type the above code into the terminal in this lesson.

This is where things get tricky and you need to think carefully!

- **Line 1** will have displayed the error as output and put nothing into the redirected-to `outfile` (**line 2**), while
- **Line 3** did not display the error and captured it in the `outfile` (**line 4**).

Remember that the redirection operator `2>&1` points standard error (file descriptor `2`) at whatever standard output (file descriptor `1`) was pointed to at the time.

If you read the first line carefully, at the point `2>&1` was used, standard output was pointed at the terminal. So standard error is pointed at the terminal from there on.

After that point, standard output is redirected (with the `>` operator) to the file `outfile`.

So at the end of all this:

- The standard error of the output of the command `command_does_not_exist` points at the terminal
- The standard output points at the file `outfile`.

In the second line (`command_does_not_exist > outfile 2>&1`), what is different?

The order of redirections is changed.

Now:

- The standard output of the command `command_does_not_exist` is pointed

- The standard output of the command `command_does_not_exist` is pointed at the file `outfile`
- The redirection operator `2>&1` points file descriptor 2 (standard error) to whatever file descriptor 1 (standard output) is pointed at

So in effect, both standard out and standard error are pointed at the same file (`outfile`).

This pattern of sending all the output to a single file is seen very often, and few understand why it has to be in that order. Once you understand, you will never pause to think about which way round the operators should go again!

Differences Between Pipes and Redirects

To recap:

- A pipe passes *standard output* as the *standard input* to another command
- A *redirect* sends a channel of output to a file

A couple of other commonly used operators are worth mentioning here:

```
grep -c file < file1
```

Type the above code into the terminal in this lesson.

The `<` operator redirects standard *input* to the `grep` command from a file. In this case, it is equivalent to the `cat file1 | grep -c file` command you saw earlier.

```
echo line1 > file3
cat file3
echo line2 > file3   # Overwrites file3, replacing its contents with 'line2'
cat file3
echo line3 >> file3  # Appends to file3
cat file3
```

Type the above code into the terminal in this lesson.

- **Lines 1 and 3** above use the `>` operator, while
- **Line 5** uses the `>>` operator.

The `>` operator effectively creates the file anew whether it already exists or

not. The `>>` operator, by contrast, *appends* to the end of the file.

As a result, only `line2` and `line3` are added to `file3`.

Pipes and Redirects Quiz

1 Choose the true statements

COMPLETED 0%

1 of 2

What You Learned

- What *file redirection* is
- What *pipes* do
- The differences between *standard output* and *standard error*
- How to *redirect standard output* to the same location as *standard error* (and vice versa)
- How to *redirect standard output or standard error* (or both) to a file

What Next?

You've completed the 'Core Bash' section! Next you'll move onto the 'Scripting Bash' section, where you'll learn about the tools needed to write significant and useful bash scripts.

Exercises

1) Try a few different commands and work out what output goes to standard output and what output goes to standard error. Try triggering errors by misusing programs.

2) Write commands to redirect standard output to file descriptor `3`.

3) Research what the `|&` pipe operator does. If you can't find out, try the command `cat file2 |& grep -c file` and see what changes compared to when you ran it above.

Scripts and Startup

This lesson will cover two related subjects: Shell scripts and what happens when the shell is started up.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Shell Scripts
- The Shebang
- The Executable Flag
- The `PATH` Variable
- Startup Scripts
- Startup Explained
- When You Run Bash
- Beware
- The `source` Builtin
- Avoiding Startup Scripts
- What You Learned
- What Next?
- Exercises

You've probably come across shell scripts, which won't take long to cover, but shell **startup** is a useful but tricky topic that catches most people out at some point, and is worth understanding well.

How Important is this Lesson?

This lesson is essential to the course.

Shell Scripts

First create a folder to work in and move into it:

Terminal

A shell script is simply a collection of shell commands that can be run non-interactively. These can be quick one-off scripts that you run, or very complex programs.

The Shebang

Run this:

```
echo '#!/bin/bash' > simple_script
echo 'echo I am a script' >> simple_script
```

Type the above code into the terminal in this lesson.

You have just created a file called `simple_script` that has two lines in it. The first consists of two special characters: the hash and the exclamation mark. This is often called *shebang*, or *hashbang* to make it easier to say. When the operating system is given a file to run as a program, if it sees those two characters at the start, it knows that the file is to be run under the control of another program (or *interpreter* as shells are often called).

Now try running it:

```
./simple_script
```

Type the above code into the terminal in this lesson.

That should have failed. Before we explain why, let's understand the command.

The `./` characters at the start of the above command tells the shell that you want to run this file from within the context of the current working directory. It's followed by the filename to run.

Similarly, the `../` characters indicate that you want to run from the directory above the current working directory.

This:

```
mkdir tmp
```

```
cd tmp
../simple_script   # Call the script from the tmp subfolder


cd ..
rm -rf tmp
```

Type the above code into the terminal in this lesson.

will give you the same output as before.

The Executable Flag

The script in the above example will have failed because the file was not marked as *executable*, so you will have got an error saying permission was denied.

To correct this, run:

```
chmod +x simple_script
./simple_script
```

Type the above code into the terminal in this lesson.

The `chmod` program changes the permissions (or *modes* on a file), so that only certain users, or groups of users can *read*, *write*, or *execute* (ie run as a program) a file.

> Note: The subject of file permissions and ownership can get complex and is not covered in full here. `man chmod` is a good place to start if you are interested.

The `PATH` Variable

What happens if you don't specify the `./` and just run:

```
simple_script
```

Type the above code into the terminal in this lesson.

Either you'll get an error saying the file cannot be found, or the command might work as before.

The reason the expected output is known is because it depends on how the `PATH` variable is set up in the environment.

If you run the code below you will see your `PATH` variable:

```
echo $PATH
```

Type the above code into the terminal in this lesson.

Your output may vary. Here is an example:

```
/home/imiell/perl5/bin:/opt/local/bin:/opt/local/sbin:/usr/local/bin:/usr/
bin:/bin:/usr/sbin:/sbin:/opt/X11/bin
```

The `PATH` variable is a set of directories, separated by colons. It could be as simple as:

```
/usr/sbin:/usr/bin
```

for example.

These are the directories bash looks through to find commands, in order.

So what sets up the `PATH` variable if you did not? The answer is: bash startup scripts.

But before we discuss them, how can we make sure that simple_script can be run without using `./` at the front?

```
PATH=${PATH}:.
simple_script
```

Type the above code into the terminal in this lesson.

That's how! In the first line you set the `PATH` to itself, plus the current working directory. It then looks through all the directories that were previously set in your `PATH` variable, and then finally tries the `.`, or local folder, as we saw before.

Startup Scripts

Understanding startup scripts and environment variables are key to a lot of issues that you can end up spending a lot of time debugging! If something

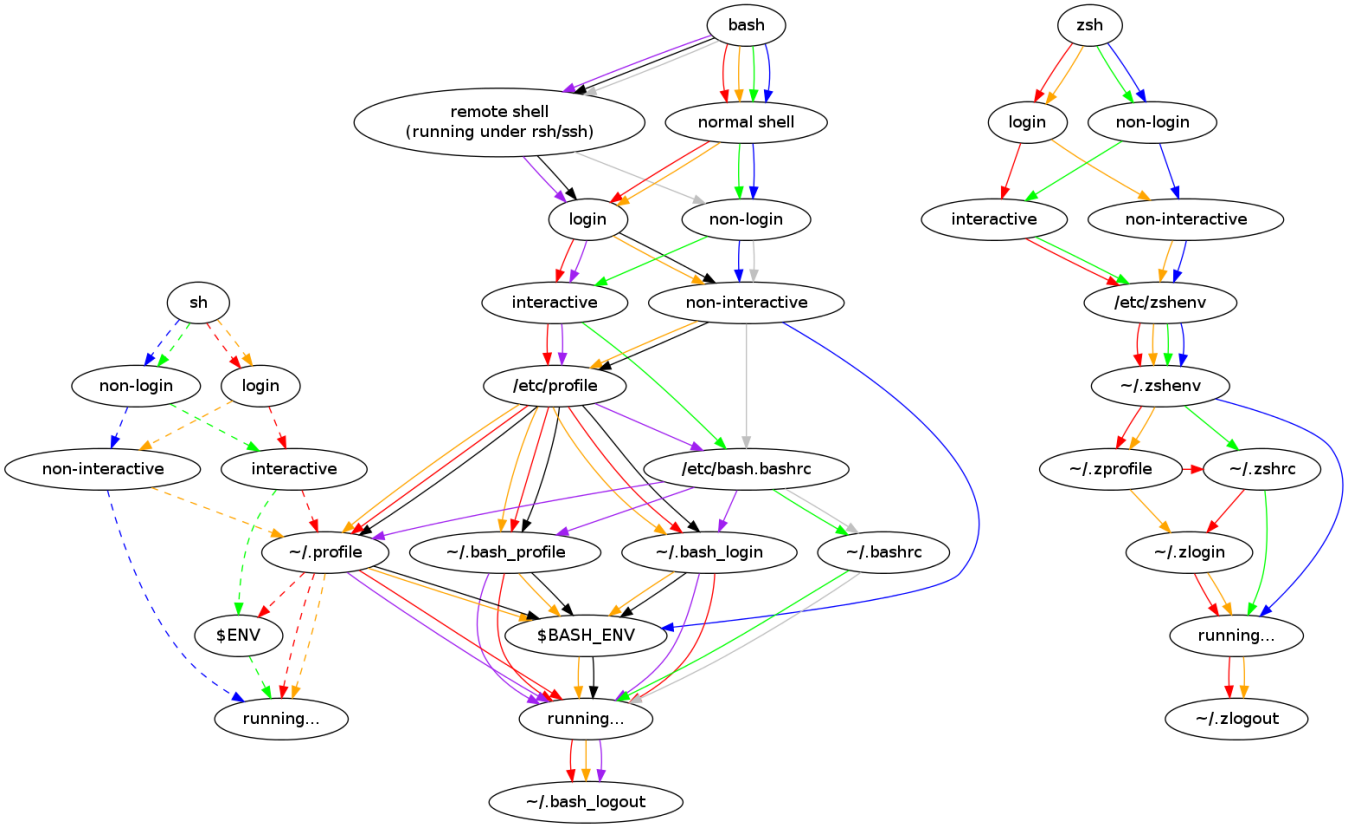
issues that you can end up spending a lot of time debugging! If something

works in one environment and not in another, the answer is often a difference in startup scripts and how they set up an environment.

Startup Explained

When bash starts up, it calls and runs a series of files to set up the environment you arrive at, at the terminal. If you've ever noticed that bash can 'pause' before giving you a prompt, it may be because the startup script is performing a command in the foreground.

Have a look at this diagram:



Yes, this can be confusing.

The diagram shows the startup script order for different shells in different contexts. Each context is shown by following a separate color through the diagram.

We are going to follow (from the top) the path from the 'bash' bubble, and ignore the 'zsh' and 'sh' paths, but it's interesting to note they have their own separate paths (in the case of 'zsh') and join up at points (in the case of 'sh' and 'bash').

At each point in this graph the shell you choose either makes a decision about which path to follow (eg whether the shell is *interactive* or not), or runs a script if the color has already been determined from these decisions.

> Note: interactive means running with a terminal attached to the process. If a script is run, and you can't type input to it, it is *non-interactive*.

We'll walk through this below, which should make things clearer.

When You Run Bash

So which path does it take when you run `bash` on the command line?

- The first decision you need to make is whether bash is running *locally* or as a *remote* shell on another server
- From there, you decide if this is a *login* or a *non-login* shell. You did not login when you ran bash, so follow 'non-login'
- The final decision is whether bash is running interactively (ie can you type things in, or is bash running a script?). You are on an interactive shell, so follow 'interactive'
- Now, whichever colour line you have followed up to this point, continue with it: those files are the ones that get run when bash is started up
- If the file does not exist, it is simply ignored

Beware

To *further* complicate things, these scripts can be made to call each other in ways that confuse you if you simply believe that diagram. So be careful!

The `source` Builtin

Now that you understand builtins, shell scripts, and environments, it's a good time to introduce another builtin: `source`.

```
MYVAR=Hello
echo 'echo $MYVAR' > simple_echo  # Write a program that outputs a variable
chmod +x simple_echo              # Make the script executable
./simple_echo                     # Call the script, which outputs nothing as MYVAR is not ex
source simple_echo                # source the script, which actually outputs the MYVAR varia
```

```
source simple_echo              # source the script, which actually outputs the MYVAR varia
```

Type the above code into the terminal in this lesson.

From the above example, you can see that the source runs the script from within the same shell context.

So if you put just `exit` within the code you run, then a `source`d script will exit the shell you are in, rather than the script that is being run!

If you didn't follow what the above script did when sourced, do it again, and think carefully about what each command is doing.

> Note: Most shell scripts have a `.sh` suffix, but this is not required - the OS does not care or take any notice of the suffix.

Avoiding Startup Scripts

It's often useful to start up bash while avoiding all these startup scripts, to get bash in as raw a state as possible. When debugging, or testing core bash functionality, this can be invaluable.

To acheive that, run this:

```
env -i bash --noprofile --norc
```

Type the above code into the terminal in this lesson.

`env` is a program that works on the environment. The effect of the `-i` flag is to remove the environment variables from the command you run. This means that exported variables will not get inherited by the bash program we are running. We're running the bash program itself with two flags. The `--noprofile` flag tells bash not to source the system-wide bash startup files, and the `--norc` tells bash not to source the personal ones present in your home folder.

The end effect of this is that your shell has a very minimal set of variables available:

```
env
```

Type the above code into the terminal in this lesson.

Type the above code into the terminal in this lesson.

Because you 'emptied' the environment, the number of variables you should see will be very small.

Scripts and Startup Quiz

1 Bash scripts must be suffixed with `.sh`. True or false?

COMPLETED 0%

1 of 2

What You Learned

- What the 'shebang' is
- How to create and run a shell script
- The significance of the `PATH` environment variable
- What happens when bash starts up
- What the builtin `source` does

What Next?

Next you look at **command substitution**, which allows you to easily capture the output of commands to variables for later use.

Exercises

1) Go through all the scripts that your bash session went through on startup in the terminal above. Read through them and try and understand what they're

doing. If you don't understand parts of them, try and figure out what's going on by reading `man bash`.

2) Go through the other files in that diagram that exist in the terminal above. Do as per 1).

Command Substitution

In this lesson, you'll learn about command substitution and other ways of substituting commands.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Command Substitution Example
- The Two Command Substitution Methods
  - The 'Dollar-Bracket' Method: `$()`
  - The 'Backtick' Method
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

The contents of this lesson are vital if you intend to write or work on shell scripts.

Command Substitution Example

When writing bash scripts you often want to take the standard output of one command and 'drop' it into the script as though you had written that into it.

An example may help illustrate this idea. Type these commands:

```
hostname                            # 'hostname' outputs the name of the host
echo 'My hostname is: $(hostname)'  # Single quotes do not call the hostname command
echo "My hostname is: $(hostname)"  # Double quotes do, similar to variable dereferencing
```

Type the above code into the terminal in this lesson.

Terminal

If those lines are placed in a script, it will output the hostname of the host the script is running on. This can make your script much more dynamic. You can set variables based on the output of commands, add debug, and so on, just as with any other programming language.

You may have noticed that if wrapped in single quotes, the special meaning of the `$` sign is ignored again!

The Two Command Substitution Methods

There are two ways to do command substitution:

```
echo "My hostname is: `hostname`"   # backticks method
echo "My hostname is: $(hostname)"  # dollar-bracket method
```

Type the above code into the terminal in this lesson.

These give the same output and the backticks perform the same function. So which should you use?

The 'Dollar-Bracket' Method: `$()`

Type this:

```
mkdir tmp
cd tmp
echo $(touch $(ls ..))
cd -
rm -rf tmp
```

Type the above code into the terminal in this lesson.

What happened there?

- In **lines 1 and 2** you created and moved into a folder
- This temporary folder is cleaned up by **lines 4-5**
- **Line 3** is best read from the innermost parentheses outwards
    - The `ls ..` command is run in the innermost parentheses. This outputs the contents of the parent directory

- This output is substituted in over the `$(ls ..)`. The 'words' returned are placed as the arguments to the `touch` command. The `touch` command creates a set of empty files, based on the list of the parent directory's contents
- The `echo` command takes the output of the above substitution, in this case nothing, as touch does not produce any output

So, in summary:

- **Line 3** outputs the list of files of the parent directory
- Those filenames are also created locally as empty files

Line 3 is an example of how subcommands can be nested. As you can see, the nesting is simple - just place a command wrapped inside a `$()` inside another command wrapped inside a `$()` and bash will then substitute the output of each command from the inside out for you in the appropriate order.

Now let's look at the equivalent code with backticks.

The 'Backtick' Method

Type this out:

```
echo `touch \`ls ..\``
cd ..
```

Type the above code into the terminal in this lesson.

To nest the **backtick** version, you have to *escape* the inner backtick with a backslash, so bash knows which level the backtick should be interpreted at.

To demonstrate the difficulty here, a simple example suffices:

```
echo $(echo hello1 $(echo hello2))   # Nesting echoes
echo `echo hello1 `echo hello2``     # Nesting echoes with backticks fails
echo `echo hello1 \`echo hello2\``   # Nesting echoes with backticks and backslashes ok
```

Type the above code into the terminal in this lesson.

For historical reasons, the backtick form is still very popular, but I prefer the `$()` form because of the simplicity of managing nesting. You need to be aware of both, though, if you are looking at others' code!

If you want to see how messy things can get, compare these two lines:

```
echo `echo \`echo \\\`echo inside\\\`\``
echo $(echo $(echo $(echo inside)))
```

Type the above code into the terminal in this lesson.

and consider which one is easier to read (and write)!

What You Learned

- What *command substitution* is
- How it relates to quoting
- The two *command substitution* methods
- Why one method is generally preferred over the other

What Next?

Next you will cover **tests**, which allow you to use what you've learned so far to make your bash code conditional in a flexible and dynamic way.

Exercises

1) Try various command subsitution commands, and plug in variables and quotes to see what happens.

2) Explain why three backslashes are required in the last example.

Tests

In this lesson, you'll learn some of the pitfalls and rules of thumb for practical usage of tests. You'll learn about how bash tests can be written, binary, unary, and logical operators, and if statements.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- What Are Bash Tests?
- What is ‘ `[` ’, Really?
- Logical Operators
- The `[[` Operator
- Unary and Binary Operators
- Types
- `if` Statements
- Bare `if` Statements
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Tests are a fundamental part of bash scripting, whether it’s on the command line in one-liners, or in much larger scripts or chains of commands.

What Are Bash Tests?

A test in bash is not a test that your program works. It’s a way of writing an expression that can be true or false.

Tests in bash are constructs that allow you to implement *conditional expressions*. They use square brackets (ie `[` and `]`) to enclose what is being tested.

For example, the simplest tests might be:

```
[ 1 = 0 ]   # Test that should fail
echo $?     # Non-zero output means 'un-true'
[ 1 = 1 ]   # Test that should succeed
echo $?     # Zero output means 'true'
```

Type the above code into the terminal in this lesson.

Terminal

> Note: The `echo $?` command above is a little mystifying at this stage if you've not seen it before. We will cover it in more depth in a lesson later in this 'Scripting Bash' section of the course. For now, all you need to understand is this: the `$?` variable is a special variable that gives you a number telling you the result of the last-executed command. If it returned true, the number will (usually) be `0'. If it didn't, the number will (usually) *not* be '0'.

Things get more interesting if you try and compare values in your tests. Think about what this will output before typing it in:

```
A=1
[ $A = 1 ]
echo $?
[ $A == 1 ]   # Does a double equals sign make any difference?
echo $?
[ $A = 2 ]
echo $?
```

Type the above code into the terminal in this lesson.

A single equals sign works just the same as a double equals sign. Generally I prefer the double one so it does not get confused with variable assignment.

What is '`[`', Really?

It is worth noting that `[` is in fact a builtin, as well as (very often) a program.

```
which [      # Tells you where the '[' program is
type [       # Tells you that '[' is also a builtin
```

Type the above code into the terminal in this lesson.

and that `[` and `test` are synonymous:

```
which test
man test    # Hit q to get out of the manual page.
man [       # Takes you to the same page.
```

Type the above code into the terminal in this lesson.

> Note: `which` is a program (not a builtin!) that tells you where a program can be found on the system.

This is why a space is required after the `[`. The `[` is a *separate command* and spacing is how bash determines where one command ends and another begins.

Logical Operators

What do you expect the output of this to be?

```
( [ 1 = 1 ] || [ ! '0' = '0' ] ) && [ '2' = '2' ]
echo $?
```

Type the above code into the terminal in this lesson.

Similar to other languages, '!' means 'not', `||` means 'or', `&&` means 'and' and items within '()' are evaluated first.

Note that to combine the binary operators `||` and `&&` you need to have separate `[` and `]` pairs.

From this point forward in this course I will use logical operators (particularly `&&`) where it's appropriate to. This should get you used to the idiom.

If you want to do everything in *one* set of braces, you can run:

```
[  1 = 1 -o  ! '0' = '0'  -a  '2' = '2' ]
echo $?
```

Type the above code into the terminal in this lesson.

You can use `-o` as an 'or' operator within the square brackets, `-a` for 'and' and so on. But you can't use `(` and `)` to group within them.

If you're not confused yet, you might be soon! If you are, try and re-read the above until you get it.

The `[[` Operator

The `[[` operator is very similar to the 'test' operator with *two* square brackets instead of one:

```
[[ 1 = 1 ]]
echo $?
```

Type the above code into the terminal in this lesson.

This confused me a lot for some time! What is the difference between `[` and `[[` if they produce such similar output?

The differences between `[[` and `[` are relatively subtle. Type these lines to see examples:

```
unset DOESNOTEXIST
[ ${DOESNOTEXIST} = '' ]
echo $?
[[ ${DOESNOTEXIST} = '' ]]
echo $?
[ x${DOESNOTEXIST} = x ]
echo $?
```

Type the above code into the terminal in this lesson.

- **Line 1** above should error because the variable `DOESNOTEXIST` does not exist
- Bash processes that variable in **line 2**, and ends up running: `[ = '' ]`
- This makes no sense to bash, so it complains! It's expecting something on the left hand side of the empty quotes
- **Line 3** shows that it errored by returning a non-zero exit code

- **Line 4** (which uses the double brackets `[[`) tolerates the fact that the variable does not exist, and treats it as the empty string. It therefore resolves to: `[ '' = '' ]`
- And does not error (**line 5**), returning `0`.
- **Line 6** acts as a workaround. By placing an `x` on both sides of the equation, the code ensures that *something* gets placed on the left hand side: `[ x = x ]`
- which returns 'true' (**line** 7 returns `0`).

You can frequently come across code like this:

```
[[ "x$DOESNOTEXIST" = "x" ]]
```

where users have put quotes on both sides *as well as* an `x` and put in double brackets. Only one of these protections is needed, but people get used to adding them to their bash scripts as a superstition. And it doesn't seem to do any harm.

Once again, you can see understanding how quotes work is critical to bash mastery!

Oh, and `[[` doesn't like the `-a` (and) and `-o` (or) operators.

So `[[` can handle some edge cases that are not handled the same way when using `[`. There are some other differences, but I won't cover them here.

> Note: If you want to understand more, go to http://serverfault.com/questions/52034/what-is-the-difference-between-double-and-single-square-brackets-in-bash

Confused?

You're not alone. In practice, I follow most style guides and always use `[[` until there is a good reason not to.

If I come across some tricky logic in code I need to understand, I just look it up there and then, usually in the bash man page.

Unary and Binary Operators

There are other shortcuts related to `test` (and its variants) that it's worth knowing about. These take a single argument:

```
echo $PWD
[ -z "$PWD" ]            # -z is a unary operator that takes one argument
echo $?                  # Returns false, as PWD has content
unset DOESNOTEXIST
[ -z "$DOESNOTEXIST" ]
echo $?                  # Returns true, as "$DOESNOTEXIST" is evaluated to empty
[ -z ]                   # No argument?
echo $?                  # Returns true...
```

If your `$PWD` environment variable is set (it usually is), then the -z will return `false`. This is because `-z` returns true only if the argument is an empty string. Interestingly, this test is OK with no argument! Just another confusing point about tests...

There are quite a few unary operators so I won't cover them all here. The ones I use most often are `-a` and `-d`:

```
touch tests_file   # Create tests file
mkdir tests_dir
[ -a tests_file ]  # -a returns true if file exists
echo $?
[ -d tests_file ]  # -d returns false if the directory does not exist
echo $?            # A normal file is not a directory
[ -a tests_dir ]   # A directory is a type of file, so returns true
echo $?
[ -d tests_dir ]   # -d returns true if directory exists
echo $?
```

Type the above code into the terminal in this lesson.

These are called 'unary operators' (because they take one argument).

There are many of these unary operators, but the differences between them are useful only in the rare cases when you need them. Generally I just use `-d`, `-a`, and `-z` and look up the others when I need something else.

We'll cover 'binary operators', which work on two arguments, while covering types in bash.

Types

Type-safety (if you're familiar with that concept from other languages) does not come up often in bash as an issue. But it is still significant. Try and work out what's going on here:

```
[ 10 < 2 ]       # Throws an error, as single bracket tests don't handle '<'
echo $?
[ '10' < '2' ]   # Quotes do not help
echo $?
[[ 10 < 2 ]]     # Double bracket tests work
echo $?          # But it treats the input as strings, not integers
[[ '10' < '2' ]] # Same result when quoted as a string
echo $?
```

Type the above code into the terminal in this lesson.

From this you should be able to work out that the `<` operator expects strings, and that this is another way `[[` protects you from the dangers of using `[`.

If you can't work it out, then re-run the above and play with it until it makes sense to you!

Then run this:

```
[ 10 -lt 2 ]      # The -lt operator works in single brackets
echo $?
[ '10' -lt '2' ]
echo $?           # Still treats arguments as integers when quoted
[ 1 -lt 2 ]
echo $?
[ 10 -gt 1 ]
echo $?
[ 1 -eq 1 ]
echo $?
[ 1 -ne 1 ]
echo $?
```

Type the above code into the terminal in this lesson.

The binary operators used above are: `-lt` (less than), `-gt` (greater than), `-eq` (equals), and `-ne` (not equals). They deal happily with integers in single bracket tests, even when quoted.

`if` Statements

Now you understand tests, `if` statements will be easy!

Type this:

```
if [[ 10 -lt 2 ]]
then
  echo 'does not compute'
elif [[ 10 -gt 2 ]]
then
  echo 'computes'
else
  echo 'neither greater than, or less than, so must be equal'
fi
```

Type the above code into the terminal in this lesson.

`if` statements consist of:

- A test
- The word `then`
  - If the `if` test returned 'true' it runs the commands after the `then`
- If the `if` test returned false, it will
  - Drop to the next `elif` statement if there is another test
  - Or drop to the `else` block if there are no more tests
- Finally, the if block is closed with the `fi` string

The `else` or `elif` blocks are not required for the `if` statement to work. For example, this will also work:

```
if [[ 10 -lt 2 ]]
then
  echo 'does not compute'
fi
```

Type the above code into the terminal in this lesson.

Bare `if` Statements

It's easy to forget that the `if` statement in bash does not need angle brackets at all. If the code between the `then` and the `if` is a bash command, then it will trigger if the exit code of the command was 'true':

What will this output? No cheating!

```
if grep not_there /dev/null
then
    echo there
else
    echo not_there
```

```
fi
```

Type the above code into the terminal in this lesson.

Line 1 has an `if` statement with a 'bare' `grep` command. If the exit code of that `grep` command is zero (ie a match was found) then it outputs `there`. Otherwise, it outputs `not there`.

Tests Quiz

1 Which of these will error if run?

COMPLETED 0%

1 of 2

What You Learned

We covered quite a lot in this lesson!

- What a *test* is in bash

- How to compare values within a *test*
- What the program `[` is
- How to perform logic operations with *tests*
- Some differences between `[` and `[[`
- The difference between *unary* and *binary operators*
- How types can matter in bash, and how to compare them
- `if` statements and *tests*

What Next?

Next you will cover another fundamental aspect of bash programming: **loops**.

Exercises

1) Research all the unary operators, and try using them (see `man bash`)

2) Write a script to check whether key files and directories are in their correct place.

3) Use the `find` and `wc` commands to count the number of files available in the lesson's terminal and perform different actions if the number is higher or lower than what you expect.

Loops

In this lesson, you will cover for loops, case statements, and while loops in bash. This lesson will quickly take you through the various forms of looping that you might come across, including 'while' loops, 'case' statements, 'for' loops, and 'C-style' loops.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- `for` Loops
- `while` Loops in Bash
- `case` Statements
- `case` Statements and Command Line Options
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

For and **while** loops are a basic construct in most programming languages, and bash is no exception.

`for` Loops

First you're going to run a for loop in a 'traditional' way:

```
for (( i=0; i < 20; i++ ))
do
  echo $i
  echo $i > file${i}.txt
done
ls
```

Type the above code into the terminal in this lesson.

Terminal

- You just created twenty files, each with a number in them using a `for` loop in the 'C' language style (**line 1**)
- Note there's no `$` sign involved in the variable when it's in the double parentheses!
- **Line 3** `echo`es the value of `i` in each iteration
- **line 4** redirects that value to a file called `file{$i}.txt`, where `${i}` is replaced with its value in that iteration
- **Line 6** shows the listing of files created

```
for f in $(ls *txt)
do
    echo "File $f contains: $(cat $f)"
done
```

Type the above code into the terminal in this lesson.

It's our old friend the command substitution! The command substitution lists all the files we have.

This `for` loop uses the `in` keyword to separate the variable each iteration will assign to `f` and the list to take items from. Here bash evaluates the output of the `ls` command and uses that as the list, but we could have written something like:

```
for f in file1.txt file2.txt file3.txt
do
    echo "File $f contains: $(cat $f)"
done
```

Type the above code into the terminal in this lesson.

with a similar effect.

`while` Loops in Bash

While loops also exist in bash. Try and work out what's going on in this trivial example:

```
n=0
while [[ ! -a newfile ]]
do
    ((n++))
    echo "In iteration $n"
    if [[ $(cat file${n}.txt) == "15" ]]
    then
            touch newfile
    fi
done
```

Type the above code into the terminal in this lesson.

- **Line 1** creates an `n` variable
- **Line 2-3** creates a *while* loop that increments the `n` variable on each iteration (**line 4**)
- **Line 5** output which iteration the loop is on
- If the file with the name `file${n}.txt` (where the `${n}` is replaced by the value of the variable `n` at the time) contains the string `15` (**line 6**), then a `newfile` file is created
- The while loop condition on **line 2** finishes when this `newfile` file exists.

I often use while loops in the following 'infinite loop' form when running quick scripts on the command line:

```
n=0
while true
do
    sleep 1
    ((n++))
    echo $n seconds have passed
    if [[ $n -eq 60 ]]           # When n reaches 60...
    then
        break                    # Break out of the while loop
    fi
done
```

Type the above code into the terminal in this lesson.

The above code is an 'infinite loop' because the while condition is `true`, so will never break out by itself. The `break` statement on **line 11**, if reached, will exit the while loop.

`case` Statements

`case` Statements

Case statements may also be familiar from other languages. In bash, they're most frequently used when processing command-line arguments within a script.

Before you look at a realistic case statement, type in this trivial one:

```
a=1                                          # Initialize 'a' variable
case "$a" in                                 # Start case statement
1) echo 'a is 1'; echo 'ok1';;               # If a == 1, then output ok1
2) echo 'a is 2'; echo 'ok2';;               # If a == 2, then output ok2
*) echo 'a is unmatched'; echo 'failure';; # If nothing was matched, output 'failure'
esac                                         # End case statement
```

Type the above code into the terminal in this lesson.

Try triggering the `a is 2` case, and the `a is unmatched` case.

There are a few of new bits of syntax you may not have seen before.

- The double semi-colons `;;` indicate that the next matching case is coming (rather than just another statement, as indicated by a single semi-colon)
- The `1)` indicates what the `case` value `$a` should match. These values follow the globbing rules (so `*` will match anything)
    - Try adding quotes around the values, or glob values, or matching a longer string with spaces
- The `esac` indicates the `case` statement is finished

`case` Statements and Command Line Options

`case` statements are most often seen in the context of processing command-line options within shell scripts. There is a helper `builtin` just for this purpose: `getopts`.

Now you will write a more realistic example, and more like what is seen in 'real' shell scripts that uses `getopts`.

Create a file `case.sh` to try out a `case` statement with `getopts`:

```
cat > case.sh << 'EOF'
```

```
#!/bin/bash
while getopts "ab:c" opt
do
    case "$opt" in
    a) echo '-a invoked';;
    b) echo "-b invoked with argument: ${OPTARG}";;
    c) echo '-c invoked';;
    esac
done
EOF
chmod +x case.sh
```

Type the above code into the terminal in this lesson.

- **Line 1** creates a file `case.sh` using a *here doc* (see here if you want to learn about these now)
- **Line 2** tells the system that this is a bash script
- **Line 3** Starts the while loop, calling the builtin `getopts`. The builtin returns true while there are more flags to process, and sets the `opt` variable to the flag being processed on this iteration. The string `ab:c` is the specification of the flags, where the `:` indicates that the previous flag mentioned takes an argument
- **Line 5** Starts a `case` statement based on the `opt` value set in this iteration
- **Line 6** deals with the `-a` flag. Note that the `-` sign is removed by `getopts` when passing into `opt`
- **Line 7** deals with the `-b` flag, which expects an argument. The argument is placed in the `OPTARG` variable by `getopts`
- **Line 8** deals with the `-a` flag

Run the above with various combinations and try and understand what's happening:

```
./case.sh -a
./case.sh -b
./case.sh -b "an argument"
./case.sh -a -b -c
./case.sh
```

Type the above code into the terminal in this lesson.

This is how many bash scripts pick up arguments from the command line and process them.

Loops Quiz

1 How do you get out of a `while` loop while it is running?

COMPLETED 0%

1 of 2

What You Learned

You've now covered the main methods of looping in bash. Nothing about looping in bash should come as a big surprise in future!

What Next?

Next you will learn about **exit codes**, which will power up your ability to write neater bash code and better scripts.

Exercises

1) Find a real program that uses `getopts` to process arguments and figure out what it's doing.

2) Write a while loop to check where a file exists every few seconds. When it does, break out of the loop with a message.

Exit Codes

In this lesson, you will find out what an exit code is, how you can set one in a script and a function, some conventions around exit codes, and other 'special' parameters in bash.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- What Is An Exit Code?
- Standard Exit Codes
- Exit Codes and `if` Statements
- Setting Your Own Exit Code
- Other Special Parameters
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Most bash scripts won't be completely comprehensible or safely written unless **exit codes** are understood.

What Is An Exit Code?

After you run a command, function or builtin, a special variable is set that tells you what the result of that command was. If you're familiar with HTTP codes like 200 or 404, this is a similar concept to that.

To take a simple example, type this in:

```
ls
echo $?
doesnotexist
echo $?
```

Type the above code into the terminal in this lesson.

Terminal

When that special variable `$?` is set to `0` it means that the previous command completed successfully (**line 2**). When the special variable `$?` is set to non-zero (in this case `127` it means that it did not complete successfully.

Standard Exit Codes

There are guidelines for exit codes for those that want to follow standards. Be aware that not all programs follow these standards (`grep` is the most common example of a non-standard program, as you will learn)!

Some key ones are:

- `0` - 'command successfully run' (OK)
- `1` - Used when there is an error but no specific number reserved to indicate what it was (general error)
- `2` - Misuse of shell `builtin` command
- `126` - Permission problem or command is not executable
- `127` - No file found matching this command
- `128` - Invalid exit argument given (eg exit 1.76)
- `128+n` - Process killed with signal `n`, eg 130 = terminated with signal 2

> Note: Signals will be covered in part 4

Since codes `3`-`125` are not generally reserved, you might use them for your own purposes in your applications.

Exit Codes and `if` Statements

So far so simple, but unfortunately (and because they are useful) exit codes can be used for many different reasons, not just to tell you whether the command completed successfully or not. Just as with exit codes in HTTP, the application can use exit codes to indicate something went wrong, or it can return a '200 OK' and give you a message directly.

Try to predict the output of this:

```
echo 'grepme' > afile.txt
grep not_there afile.txt
echo $?
```

Type the above code into the terminal in this lesson.

Did you expect that?

- **Line 1** created a file called `afile.txt`
- On **line 2** `grep` finished successfully (there was no segmentation or memory fault, memory exhaustion, it was not killed etc) but no lines were matched, and it returned `1` as an exit code (**line 3**).

`grep` uses the exit code `0` to mean 'matched', and the exit code `1` to mean 'not matched'.

In one way this is great, because you can write if statements like this:

```
if grep grepme afile.txt
then
    echo 'matched!'
fi
```

Type the above code into the terminal in this lesson.

In the above code, the `if` statement on **line 1** returns a zero exit code (true) if the `grep` command matched any lines, and outputs the echo on **line 3**.

While that is a great feature of `grep`, it means that you cannot be sure about what an exit code might mean about a particular program's termination. I have to look up the meaning of `grep`'s exit code nearly every time, and if I use a program's exit code I usually make sure to do a few tests first to be sure I know what is going to happen!

Setting Your Own Exit Code

If you are writing a script, you can set the exit code of the function by using the `exit` builtin. You can simulate this simply by entering a new bash process and then exiting from it.

```
bash
exit 67    # Exit the just-entered bash shell with exit code 67
echo $?    # Retrieve the exit code
```

Type the above code into the terminal in this lesson.

Line 3 above should output the value of the exit code you quit the bash shell you set (in this case, `67`.

If you are writing a function, you can set the exit code of the function by using the `return` builtin.

Type this:

```
function trycmd {
    $1
    if [[ $? -eq 127 ]]
    then
        echo 'What are you doing?'
        return 1
    fi
}
trycmd ls
trycmd doesnotexit
```

Type the above code into the terminal in this lesson.

- **Lines 1-8** create a function called `trycmd` that runs only the command passed in as the first argument on **line 2**. The `$1` is replaced by bash with the command, which bash then runs.
- **Line 3** compares the exit code of the command just run to `127` (the error code for 'command not found') and returns with a `1` error code **line 6** and a message asking what the user is doing (**line** 7).
- The last two lines (**lines 9-10**) are example invocations of `trycmd`, one with `ls`, and one with `doesnotexist`.

Other Special Parameters

The variable `$?` is an example of a **special parameter**. I'm not sure why they are called special parameters and not special variables, but it is perhaps to do with the fact that they are considered alongside the normal parameters of functions and scripts (`$1`, `$2` etc) as automatically assigned variables within these contexts.

Two of the most important are used below. As an exercise, try and figure out what they are from context:

```
ps -ef | grep bash | grep $$
sleep 999 &                    # The & here runs the command in the background
echo $!
```

Type the above code into the terminal in this lesson.

> Note: If you want to know more about background jobs (triggered by the `&` above), then see here.

If you're still stuck, have a look at the bash man page by running `man bash`.

Exit Codes Quiz

1 `$?` is an example of a/an

COMPLETED 0%

1 of 2

What You Learned

- What an *exit code* is
- Some standard *exit codes* and their significance

- Not all applications use *exit codes* in the same way
- How tests and *exit codes* work together
- Some *special parameters*

What Next?

Next you will learn about bash **options**, and the `set` builtin.

Exercises

1) Look up under what circumstances git returns a non-zero exit code.

2) Look up all the 'special parameters' and see what they do. Play with them. Research under what circumstances you might want to use them.

The 'set' Builtin

In this lesson you will become familiar with the `set` builtin, which allows you to manipulate these options within your scripts. You will cover the 'set' builtin, what POSIX is, and some useful options to set when scripting.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Running `set`
- `set` vs `env`
- Useful Options for Scripting
- Flags With `set` Instead of Names
- The `pipefail` Option, Exit Codes and Pipelines
- `set` vs `shopt`
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

When using bash it is very important to understand what its **options** are, how to set them, and how this can affect the running of your scripts.

Setting options is not absolutely core material, but once you've used bash for a while, understanding them becomes important. It's also good revision material for some of the concepts covered previously regarding exported vs. non-exported variables.

Running `set`

Start by running `set` on its own:

```
set
```

Type the above code into the terminal in this lesson.

Terminal

This will produce a stream of output that represents the state of your shell. In the normal case, you will see all the variables and functions set in your environment.

But my bash man page says:

> Without options, the name and value of each shell variable are displayed in a format that can be reused as input for setting or resetting the currently-set variables. Read-only variables cannot be reset. In posix mode, only shell variables are listed.
>
> — bash man page

> Note: The **Portable Operating System Interface(POSIX)** is a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems.

Can you work out from your `set` output whether you are in posix mode?

It is likely that you are not. If so, type:

```
bash
set -o posix
set
exit
```

Type the above code into the terminal in this lesson.

and you will observe that the output no longer has functions in it.

The `-o` in **line 2** switched on the posix option in your bash shell. The same command with `+o` will switch it off. I have trouble remembering which is 'on' and which is 'off' almost every time!

> Note: If you did not have functions, then either no functions were set, or

> Note: If you did not have functions, then either no functions were set, or you were in posix mode already!

The commands above put you in a fresh bash shell so that we would revert to the previous state.

To see how all your options are set type this:

```
set -o
```

Type the above code into the terminal in this lesson.

and you will see the current state of all your options.

What you see are all the options bash can set. One of the exercises below is to try to understand what they all mean, but in this section we're only going to focus on a couple that I use all the time.

`set` vs `env`

One thing that can confuse people is that the output of `set` is similar to the output of `env`, but different.

```
set
env
```

Type the above code into the terminal in this lesson.

The difference is that *exported* variables are shown by env, not all the variables set in the shell.

Useful Options for Scripting

Where set becomes really useful to understand is in scripting.

For example, I set these three up every time I start writing a shell script:

```
set -o errexit
set -o xtrace
set -o nounset
```

Type the above code into the terminal in this lesson.

Although you don't need to be in a script for them to work.

- The `errexit` option tells bash to exit the script if any command fails
- The `xtrace` option outputs each command as it is being run. This is really useful for seeing what command was actually run if (for example) you are using variables within your commands. It also helps you see the order in which commands are being run
- The `nounset` option gets bash to throw an error if a variable is not set when it is referred to

To see how this works in practice, first create a file:

```
echo '#!/bin/bash
set -o errexit
set -o xtrace
set -o nounset
pwd
cd $HOME
cd -
echo $DOESNOTEXIST
echo "should not get here"' > ascript.sh
```

Type the above code into the terminal in this lesson.

- The `ascript.sh` file you just created (by a redirect on **line 9**) declares itself as a bash script on **line 1**
- It then sets the three flags we mentioned above. Then it outputs the current working directory (`pwd` on **line 5**)
- **Lines 6 and** 7 move to the `$HOME` folder and back again
- **line 8** tries to `echo` a variable that does not exist

After that file is created, make it executable and run it:

```
chmod +x ascript.sh
./ascript.sh
```

Type the above code into the terminal in this lesson.

When the script is run, you should see the commands after the `xtrace` option is set outputted with a `+` sign in front. Interspersed with these lines are the output that would normally be seen on the terminal without the `xtrace`

option being set.

What you should *not* have seen is the last echo line that outputs `should not get here`. Because the `DOESNOTEXIST` variable is unset, the script will have finished with an error before that line is read in. This is because the `nounset` option is set on **line 4** of the `ascript.sh` script.

You should be able to explain to someone else what's going on at each line typed in, and what the output of the above means.

Flags With `set` Instead of Names

For each `set` option, you can use a flag instead. For example, this:

```
set -e
set -x
```

Type the above code into the terminal in this lesson.

is the same as:

```
set -o errexit
set -o xtrace
```

Type the above code into the terminal in this lesson.

I generally prefer the name form rather than flag, just because it's easier to read.

When I start writing a script, I usually start with the following:

```
#!/bin/bash

set -o errexit
set -o xtrace
set -o unset

[... remainder of script ...]
```

The `unset` throws an error if a variable is unset when referenced. (The special variables `$@` and `$*` are exempt.)

The `pipefail` Option, Exit Codes and Pipelines

The `pipefail` Option, Exit Codes and Pipelines

One option worth mentioning (as it is frequently referred to) is the **pipefail** option:

```
touch afile.txt
set -o pipefail
grep notthere afile.txt | xargs
echo $?
```

Type the above code into the terminal in this lesson.

As you know, the grep that doesn't find anything to match returns an exit code of `1` (false). This output is passed to `xargs`, which always returns a zero (true).

So the question here is: what exit code should be returned? True, because `xargs` always returns true? Or false, because the grep returned a non-zero exit code?

The answer depends on the `pipefail` setting.

```
set +o pipefail
grep notthere afile.txt | xargs
echo $?
```

Type the above code into the terminal in this lesson.

When switched on (remember, `-o` is on, `+o` is off - yes, I find it confusing too), the `pipefail` option ensure that the error code of the last command returns a non-zero status. Since `grep` returns non-zero even when there's no 'error' as such, you can get surprising behaviour when using pipes and exit codes.

By default, `pipefail` is off, so the second outcome is the default one.

`set` vs `shopt`

Although we don't cover it in depth in this section, it's worth mentioning that there are two ways to set bash options from within scripts or on the command line. You can use the `set` builtin command, or the `shopt` builtin command. They both manipulate the behaviour of the shell, and differ for historical reasons. The set options are inherited, or borrowed, from other shells'

options, while the `shopt` ones (mostly) originated in bash.

Just to demonstrate one option that you might find useful, the `globstar` option allows you to use two asterisks to match all files in the local directory and all subdirectories:

```
shopt -s globstar
ls **
```

Type the above code into the terminal in this lesson.

If you have a lot of files in your subfolders, then it might take a long time to return!

'Set' Quiz

1 `set +x` switches an option 'on'. True or False?

COMPLETED 0%

1 of 3

What You Learned

- What the `set` builtin is
- How to set an *option*
- The difference between `-o` (on) and `+o` (off)
- Some of the most-used and useful *options*

What Next?

Next we cover **process substitution**, a tricky but useful concept to master for sophisticated bash scripting.

Exercises

1) Read the man page to see what all the options are. Don't worry if you don't understand it all yet, just get a feel for what's there.

2) Set up a shell with unique variables and functions in the lesson terminal and use `set` to create a script to recreate those items in a fresh bash shell.

Process Substitution

In this lesson, you will learn about process substitution, a handy way to save time and make scripts more concise and elegant. What you'll learn will also allow you to capture the output of a command into a variable so that you can use it in other contexts or access it later.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Simple Process Substitution
- The `<()` operator
- The `>()` operator
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

I spent years reading and writing bash before I understood this concept, so this lesson can be skipped. However, since I learned about **process substitution**, I use it on the command line almost every day, so I recommend you learn it at some point.

Simple Process Substitution

Type this in to set files up for this lesson:

```
mkdir a
mkdir b
touch a/1 a/2  # Creates files 1 and 2 in folder a
touch b/2 b/3  # Creates files 2 and 3 in folder b
ls a
ls b
```

Type the above code into the terminal in this lesson.

Terminal

You've created two folders with slightly different contents.

Now let's say that you want to diff the output of `ls a` and `ls b` (a trivial but usefully simple example here). How would you do it?

> Note: if you are not familiar with the `diff` command, you can find an introduction to it here.

You might do it like this:

```
ls a > aout      # List the files in the folder 'a' and redirect output to the 'aout' file
ls b > bout      # List the files in the folder 'b' and redirect output to the 'bout' file
diff aout bout   # Diff the two outputs
rm aout bout     # Clean up the temporary output files
```

Type the above code into the terminal in this lesson.

That works, and there's nothing wrong with it, but typing all that out and cleaning up the files is a bit cumbersome. There's a much neater way that exposes a very useful technique.

Type this in:

```
diff <(ls a) <(ls b)
```

Type the above code into the terminal in this lesson.

That's neater, isn't it?

So what's going on?

The `<()` operator

The `<()` operator is conceptually similar to the `$()` we saw ealier. In the same way that `$()` substitutes the *output* of the process contained within it into the command, eg:

```
echo $(ls a)
```

Type the above code into the terminal in this lesson.

the `<()` operator substitutes a *file containing the output* of the process contained within it. You might need to stop and think about this for a second.

That means that this line:

```
diff <(ls a) <(ls b)
```

effectively becomes the command to diff two files, equivalent to the files `aout` and `bout` in the lines you typed in above.

So *wherever you would normally put a filename*, you can use the `<()` operator to save some time by dropping these in rather than creating files.

The `>()` operator

Can you guess what this does? It's similar to the `<()` but for me it was a lot trickier to grasp, and much more rarely seen (so feel free to skip).

See if you can work out from this line what it does:

```
tar cvf >(cat > out.tar) /tmp
```

Type the above code into the terminal in this lesson.

As with the `<()` operator, this replaces a file in a command. This time, rather than sending the output to the file, it takes *input* from the command that would normally go to that file reference, and feeds that input to the command in the operator.

Let's take a step back and think about that, because it can be hard to follow.

Normally you'd write something like this:

```
rm -f out.tar
tar cvf out.tar /tmp
```

Type the above code into the terminal in this lesson.

The command is read into bash, expanded out, and the tar command accepts two arguments: a file and a folder (`out.tar` and the `/tmp`). It tars up the

contents of the `/tmp` folder and places it in the `out.tar` file.

The difference in the previous command is that the contents that would normally be inputted into the file are instead fed into the command `cat > out.tar`.

Obviously, in this case that command is pointless - in both cases you end up with a file called `out.tar` that is a tar file.

Let's say, however, that you wanted to use a different compression scheme for your tar file. You could type this:

```
tar cvf >(gzip > out.tar.gz) /tmp
```

Type the above code into the terminal in this lesson.

which would gzip the tarfile and place it into the `out.tar.gz` file.

It can reasonably be pointed out that most versions of tar offer a gzip flag (`-z`). that does this for you. However, some versions don't (especially on minimal Linux distributions like busybox), so this can be a neat way of getting round that.

I have never had a need to use this mechanism in real life, but I've written things like this before, which are less neat (but good enough):

```
tar cvf out.tar /tmp
gzip out.tar
```

Type the above code into the terminal in this lesson.

What You Learned

- What the `<()` operator is
- What the `>()` operator is
- How to use these operators
- How `<()` differs from the `$()` operator
- How `>()` works

What Next?

Next you will cover **subshells**, and **grouping** commands more generally.

Exercises

1) Look for examples of where these operators are used on the web, and figure out what they're doing.

2) Look through your bash history on other machines where you have been using bash (by typing `history`) and see where you could have used these operators.

3) Construct a command that uses `$()`, `<()`, and `>()`.

Subshells

In this lesson, we'll go over why subshells are useful, how they are created, and contrasting them with group commands.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Subshells and Scope
- The `export` builtin
- Subshells and Redirection
- `()` vs `{}`
- Subshells and Working Directories
- What You Learned
- What Next?
- Exercises

The concept of **subshells** is not a complicated one, but can lead to a little confusion at times, and occasionally is very useful.

How Important is this Lesson?

As with process substitution, this is another concept I came to later in my bash career. It comes in handy fairly often, and an understanding of it helps deepen your bash wisdom.

```
VAR1='the original variable'
```

Type the above code into the terminal in this lesson.

Terminal

Now create a subshell:

```
(
```

Type the above code into the terminal in this lesson.

You'll notice the prompt has changed. Now try and echo something:

```
echo Inside the subshell
```

Type the above code into the terminal in this lesson.

It's not been run. This is because the subshell's instructions aren't run until the parenthesis has been closed. Next we'll try and echo the variable we created outside.

```
echo ${VAR1}
```

Type the above code into the terminal in this lesson.

Then update that variable to another value, and echo it again:

```
VAR1='the updated variable'
echo ${VAR1}
```

Type the above code into the terminal in this lesson.

And finally create another variable, before closing the subshell out:

```
VAR2='the second variable'
)
```

Type the above code into the terminal in this lesson.

So a subshell is a shell that inherits variables from the parent shell, and whose running is deferred until the parentheses are closed.

It will pay to think about the output and review the subshell commands to grasp what you just saw. Play with the commands and experiment until you're comfortable with what a subshell can and can't do.

Subshells and Scope

What happened to the variable you updated inside the subshell?

```
echo ${VAR1}
```

Type the above code into the terminal in this lesson.

Now you now know that variables can mask their parent shell's value within the subshell. This is very handy to know if you want to do something in a slightly modified environment, as you do not have to manage variables' previous values.

What happens if we try `export`ing the variable? Will that 'export' it to the parent shell?

```
( export VAR1='the first variable exported' )
echo ${VAR1}
```

Type the above code into the terminal in this lesson.

You will see that it does not. `export` does not 'export' the variable outside the enclosing parentheses.

The `export` builtin

The `export` command can cause a lot of confusion here. Let's experiment with it here to show how it works.

> Note: The next code snippet puts quotes around the `END` delimiter. We will cover why in the next part of the course.

```
cat > echoes.sh << 'END'
#!/bin/bash
echo $EXPORTED
echo $NOTEXPORTED
END
chmod +x echoes.sh
```

Type the above code into the terminal in this lesson.

The above code creates the `echoes.sh` file and makes the file executable.

Now run this. What do you think will happen?

Now run this. What do you think will happen?

```
export EXPORTED='exported'
NOTEXPORTED='exported'
./echoes.sh
```

Type the above code into the terminal in this lesson.

The first variable (`EXPORTED` on **line 1**) is seen in the outputs of the command run in the same shell `echoes.sh` because it is preceded by the `export` builtin.

The second variables (`NOTEXPORTED` on **line 2**) is not `export`ed, so is not seen by the command run from the same shell.

If the above doesn't make sense to you, you might want to revisit Part I, where `export` was covered in 'Variables'.

Subshells and Redirection

One often useful property of subshells is the fact that the code is treated as a single unit. You can therefore redirect output from a set of commands wholesale.

You might write code that looks like this:

```
echo Some output, ls to follow >> logfile
ls >> logfile
```

Type the above code into the terminal in this lesson.

which is fine for a couple of lines, but what if you have hundreds of lines like this? It leads to quite inelegant code. Instead, you can write:

```
(
echo Some output, ls to follow
ls
) >> logfile
```

Type the above code into the terminal in this lesson.

which is much neater and easier to manage.

`()` vs `{}`

One source of confusion is the difference between these two ways of grouping commands.
First, try replacing the `(` and `)` in the above listing with curlies (`{` and `}`) and see what happens.

Then, try these:

```
( echo output ) >> logfile
{ echo output } >> logfile
```

Type the above code into the terminal in this lesson.

Hmmm. Hit `CTRL+c` to get out.

The curlies need a semicolon to indicate the end.

```
{ echo output ; } >> logfile
```

Type the above code into the terminal in this lesson.

This is because the curlies are a *grouping command* and need an indication of when the command has been completed. No subshell is created. The environment, variables, current working directory, indeed the entire context are the same as the surrounding code.

Subshells and Working Directories

Another useful feature of subshells is that if you change folder, then that folder change only applies within the subshell. When you return, you go back to where you were.

Let's see this in action. First show where you are:

```
pwd
```

Type the above code into the terminal in this lesson.

Then, in a subshell, create a folder, move into it, and show you have moved with another call to pwd.

```
( mkdir lbthw_subshells_2
```

```
cd lbthw_subshells_2
pwd )
```

Type the above code into the terminal in this lesson.

Now the subshell has run, and without doing anything other than `cd`ing to a new folder, we are returned to the folder we were in before the subshell started:

```
pwd
```

Type the above code into the terminal in this lesson.

This is a very useful feature of subshells when you're scripting, to save repeated `cd` and `cd -` commands.

Subshells Quiz

Q What are the benefits of subshells?

COMPLETED 0%

1 of 1

What You Learned

- How to create a *subshell*
- The difference between `()` and `{}`
- How variable scoping works in *subshells* and *grouping* commands
- Redirection in *subshells* and groups

What Next?

Next you will look at a common challenge when dealing with files and for loops: the **internal field separator**.

Exercises

1) Show that the grouping curly braces (`{` and `}`) affect the shell's working directory outside the enclosed code, while a subshell does not.

2) Work out what the `BASH_SUBSHELL` variable does.

The Internal Field Separator

In this lesson we will cover how and when to use the 'IFS' variable to protect your scripts from bugs, as well as other techniques available to achieve similar ends.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Files With Spaces
- The IFS Shell Variable
  - Code Explanation
- The Null Byte as Separator
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Learning this concept will save you a great deal of time trying to figure out why your for loop is not working as expected, and will help you write more correct bash scripts that won't fail.

Files With Spaces

Create a couple of files with spaces in their names:

```
echo file1 created > "Spaces in filename1.txt"
cat "Spaces in filename1.txt"
echo file2 created > "Spaces in filename2.txt"
cat "Spaces in filename2.txt"
```

Type the above code into the terminal in this lesson.

Terminal

Note that you have to quote the filename to get the spaces in. Without the quotes, bash treats the space as a token separator. This means that it would treat the redirection as going to the file `Spaces` and not know what to do with the `in` and `filenameN.txt` tokens.

Now if you write a `for` loop over these files that just runs `ls` on each file in turn, you might not get what you expect. Type this in and see what happens:

```
for f in $(ls)
do
    ls $f
done
```

Type the above code into the terminal in this lesson.

Hmmm. The for loop has treated every word in the filenames as a separate token to look for with `ls`.

In other words, bash has treated each space as a 'field separator'. Normally this is fine, as our for loops have items separated by spaces, like this:

```
for f in 1 2 3 4
do
    echo $f
done
```

Type the above code into the terminal in this lesson.

However, here we want the spaces to be ignored. And we can control this by setting the `IFS` shell variable.

The IFS Shell Variable

```
echo $IFS
```

Type the above code into the terminal in this lesson.

If you retained the default, then you will have seen nothing in the output, which isn't very helpful. To see how it's really set up, we can use `set`:

```
set | grep IFS
```

Type the above code into the terminal in this lesson.

You should see output like this:

```
    IFS=$' \t\n'
```

Recall that the `$` before the single quotes means that the variable is showing you special characters with the backlash escape notation. Above, the `IFS` variable is set to: space, tab, and newline.

Bash takes the characters in that variable and will treat any of them as a field separator, which means that the original for loop you wrote above will create from these files:

```
    Spaces in filename1.txt
    Spaces in filename2.txt
```

these list of items:

```
    Spaces
    in
    filename1.txt
    Spaces
    in
    filename2.txt
```

What we want to do is treat the spaces like any other character. We can do this by altering the `IFS` variable to remove the space and re-running the for loop:

```
IFS=$'\t\n'
for f in $(ls)
do
    ls $f
done
```

Type the above code into the terminal in this lesson.

This might sound obscure but it's quite common in bash to perform operations over bunches of files like this:

```
find . -type f | xargs -n1 grep somestring
```

and in these cases you may have files lying around that have spaces in their names. Then your scripts can fail in ways that it can be difficult to debug when you don't know about the `IFS` variable.

Code Explanation

The `find` program used above (as the name suggests) helps you find files.

- If you just give it a folder (as above with the '.' local directory) then it will return all files and folders under that folder
- The `-type f` flag tells find to just return files, not folders
- The `xargs` command runs the command given against the files piped in
- The `-n1` flag tells xargs to run the command once per field piped in

The Null Byte as Separator

While we're on the subject, it's worth mentioning quickly a pattern that's very commonly-used to get around the above scenario.

```
find . -type f -print0 | xargs -0 -n1 grep somestring
```

By adding the `-print0` flag, find no longer uses a new line as a field separator. It uses what's called a `NUL` byte as the separator. The `NUL` byte is literally a byte of value zero. It doesn't get displayed on the screen, but can be read by `xargs` as the separator if it's given the `-0` flag. This bypasses all sorts of challenges you might get fiddling with the `IFS` variable or dealing with spaces or other odd characters in filenames.

What You Learned

- What the `IFS` shell variable does
- How to deal with filenames containing spaces and other unusual characters
- What the `find` and `xargs` programs do
- What a `NUL` byte is

What Next?

Well done! You've made it to the end of the 'Scripting Bash' section of the course. Now you are fully equipped to write and read useful shell scripts.

The next part takes what you've learned so far and extends it to understanding in a more advanced way how bash is used day to day on the command line.

Exercises

1) Set the `IFS` variable to various characters and demonstrate how field separation works

2) Run the output of `find . -type f` through hexdump and pick out the `NUL` bytes in the stream

The Readline Library

In this lesson, you'll learn about what readline is, what bash looks like without it, and how to manipulate and find out more about its features.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Bash Without Readline
- Other Shortcuts
- Using `bind` to Show Readline Shortcuts
- Readline and Terminal Options
- What You Learned
- What Next?
- Exercises

Readline is one of those technologies that is so commonly used people don't realise it's there. In this lesson I want to make you aware of it and introduce some concepts and examples of its use so you're able to deal with any problems that arise from its use or misuse.

How Important is this Lesson?

Writing bash does not require a knowledge of readline. Understanding readline helps greatly to understand what is going on at the terminal and at the command line.

Bash Without Readline

First you're going to see what bash looks like without readline.

In your 'normal' bash shell, hit the `TAB` key twice. You should see something like this:

```
Display all 2335 possibilities? (y or n)
```

That's because bash normally has an *autocomplete* function that allows you to see what commands are available to you if you tap `TAB` twice.

Hit `n` to get out of that autocomplete.

Similarly, if you hit the up arrow key a few times, then the previously-run commands should be brought back to the command line.

Now type:

```
bash --noediting
```

Type the above code into the terminal in this lesson.

Terminal

The `--noediting` flag starts up bash without the readline library enabled. If you hit tab twice now you will see something different: the shell no longer 'sees' your tab and just sends a tab direct to the screen. Autocomplete has gone.

Autocomplete is just one of the things that the readline library gives you in the terminal. You might want to try hitting the up or down arrows as you did above to see that that no longer works as well.

Hit return to get a fresh command line, and exit your non-readline-enabled bash shell:

```
exit
```

Type the above code into the terminal in this lesson.

Other Shortcuts

There are a great many shortcuts like autocomplete available to you if readline is enabled. I'll quickly outline four of the most commonly-used of these before explaining how you can find out more.

```
echo 'some command'
```

Type the above code into the terminal in this lesson.

There should not be many surprises there. Now:

- If you hit the `up` arrow, you will see you can get the last command back on your line
- If you like, you can re-run the command, but there are other things you can do with readline before you hit return
- If you hold down the `ctrl` key and then hit `a` at the same time your cursor will return to the start of the line
- Another way of representing this 'multi-key' way of inputting is to write it like this: `\C-a`. The `\C` represents the control key, and the '`-a`' represents that the `a` key is depressed at the same time
- Now if you hit `\C-e` (`ctrl` and `e`) then your cursor has moved to the end of the line. I use these two shortcuts dozens of times a day
- Another frequently useful one is `\C-l`, which clears the screen, but leaves your command line intact
- The last one I'll show you allows you to search your history to find matching commands while you type. Hit `\C-r`, and then type `ec`. You should see the `echo` command you just ran like this:

```
(reverse-i-search)`ec': echo echo
```

- Then do it again, but keep hitting `\C-r` over and over
- You should see all the commands that have `ec` in them that you've input before (if you've only got one `echo` command in your history then you will only see one)
- As you see them you are placed at that point in your history and you can move up and down from there or just hit return to re-run if you want

Here's a table of the most frequently used shortcuts for reference:

Readling Shortcut	Effect

\C-a	Takes you to start of line
\C-e	Takes you to end of line
\C-h	Delete character
\C-l	Clear screen
\C-r	Review history

> Note: What is your **history**? Your history is a list of commands that were previously-ran in your shell. We will cover this in a later lesson.

There are many more shortcuts that you can use that readline gives you. Next I'll show you how to view these.

Using `bind` to Show Readline Shortcuts

If you type:

```
bind -p
```

Type the above code into the terminal in this lesson.

You will see a list of bindings that readline is capable of. There's a lot of them! Have a read through if you're interested, but don't worry about understanding them all yet.

If you type:

```
bind -p | grep C-a
```

Type the above code into the terminal in this lesson.

you'll pick out the 'beginning-of-line' binding you used before, and see the `\C-a` notation I showed you before.

As an exercise at this point, you might want to look for the `\C-e` and `\C-r` bindings we used previously.

If you want to look through the entirety of the `bind -p` output, then you will want to know that:

- `\M` refers to the `Meta` key (which you might also know as the `Alt` key)
- `\e` refers to the `Esc` key on your keyboard. - The 'escape' key bindings are different in that you don't hit it and another key at the same time, rather you hit it, and then hit another key afterwards
    - For example, typing the `Esc` key, and then the `?` key also tries to auto-complete the command you are typing. This is documented in the `bind -p` output as:

```
"\e?": possible-completions
```

Readline and Terminal Options

If you've looked over the possibilities that readline offers you, you might have seen the `\C-r` binding we looked at earlier:

```
"\C-r": reverse-search-history
```

You might also have seen that there is another binding that allows you to search forward through your history too:

```
"\C-s": forward-search-history
```

What often happens to me is that I hit `\C-r` over and over again, and then go too fast through the history and fly past the command I was looking for. In these cases I might try to hit `\C-s` to search forward and get to the one I missed.

Watch out though! Hitting `\C-s` to search forward through the history might well not work for you.

Why is this, if the binding is there and readline is switched on?

It's because something picked up the `\C-s` before it got to the readline library:

It's because something picked up the `\C-S` before it got to the readline library: the terminal settings.

The terminal program you are running in may have standard settings that do other things on hitting some of these shortcuts before readline gets to see it.

If you type:

```
stty -a
```

Type the above code into the terminal in this lesson.

you should get output similar to this:

```
speed 38400 baud; rows 24; columns 101; line = 0;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = M-^?; eol
2 = M-^?; swtch = <undef>;
start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^V; dis
card = ^O;
min = 1; time = 0;
-parenb -parodd -cmspar cs8 hupcl -cstopb cread -clocal -crtscts
-ignbrk brkint -ignpar -parmrk -inpck -istrip -inlcr -igncr icrnl ixon -ix
off -iuclc ixany imaxbel
iutf8
opost -olcuc -ocrnl onlcr -onocr -onlret -ofill -ofdel nl0 cr0 tab0 bs0 vt
0 ff0
isig icanon iexten echo echoe echok -echonl -noflsh -xcase -tostop -echopr
t echoctl echoke -flusho
-extproc
```

You can see on the second line there is a list of key bindings that your terminal will pick up before readline sees them. The `^` character (known as the *caret*) here represents the '`ctrl`' key that we previously represented with a `\C`.

If you think this is confusing I won't disagree. Unfortunately, in the history of Unix and Linux documenters did not stick to one way of describing these keys.

If you encounter a problem where the terminal options seem to catch a shortcut key binding before it gets to readline, then you can use the `stty` program to unset that binding. In this case, we want to unset the 'stop' binding.

If you are in the same situation, type:

```
stty stop undef
```

Type the above code into the terminal in this lesson.

Now, if you re-run `stty -a`, the list of key bindings should look like this:

```
[...]
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = M-^?; eol
2 = M-^?; swtch = <undef>;
start = ^Q; stop = <undef>; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^
V; discard = ^O;
```

where the `stop` entry now has ’`<undef>` assigned to it.

Strangely, for me `C-r` is also bound to ‘reprint’ above (`^R`). But (on my terminals at least) that gets to readline without issue as I search up the history. Why this is the case I haven’t been able to figure out. I suspect that reprint is ignored by modern terminals that don’t need to ‘reprint’ the current line.

While we are looking at this list:

```
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = M-^?; eol
2 = M-^?; swtch = <undef>;
start = ^Q; stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^V; dis
card = ^O;
```

it’s worth noting a few other key bindings that are used regularly.

First, one you may well already be familiar with is `\C-c`, which interrupts a program, terminating it:

```
sleep 99 # NOW Hit \C-c
```

Type the above code into the terminal in this lesson.

After hitting `\C-c` you should see the output:

```
^C
```

Similarly, `\C-z` suspends a program, allowing you to ‘foreground’ it again and

continue with the `fg` builtin.

```
sleep 10 # NOW hit \C-z
fg
sleep 10
```

Type the above code into the terminal in this lesson.

`\C-d` sends an 'end of file' character. It's often used to indicate to a program that input is over. If you type it on a bash shell, the bash shell you are in will close.

Finally, `\C-w` deletes the word before the cursor

These are the most commonly-used shortcuts that are picked up by the terminal before they get to the readline library.

Readline Quiz

1

`\e` refers to the

COMPLETED 0%

1 of 3

What You Learned

- What the *readline* library is

- What the *readline* library is
- What features it gives you
- How to find out more about those features
- The order in which terminal options and *readline* are considered
- Some terminal-native shortcuts
- How key combinations are represented in standard documentation

What Next?

Armed with your increased knowledge of how the *readline* library works, and terminal program settings, you are ready to tackle the tricky subject of **terminal codes**.

Exercises

1) Research the terminal options' shortcuts in the `stty -a` list and what they mean.

2) Research the inputrc file and edit it to configure how `backward-kill-word` is invoked.

Terminal Codes

In this lesson you will learn about how the terminal can be manipulated to display non-standard characters that direct it to change its appearance.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Non-Printable Characters
- Using `echo`
- `CTRL-v` Escaping
- Carriage Returns vs Line Feeds
- Hexdump
- Terminal Escape Codes
- What You Learned
- What Next?
- Exercises

Although not directly related to bash, if you spend any time at a terminal, then it will pay off to understand how the terminal works with *non-standard characters*. You've already learned about the readline library and terminal options and how certain keystrokes can be 'caught' and handled in other ways before they get to the terminal. Here we look at how other non-standard characters can be handled in the bash shell.

Non-standard characters are characters like *tab*, *newline*, *carriage return*, and even the *end of file* characters. They don't form part of words, or necessarily print anything to the screen, but they are bytes interpreted by the shell and the terminal if they get that far.

> Note: The focus here is on ANSI-standard escape codes. Rarely, you might

> come across more complex escapes for specific terminal contexts, but this is beyond the scope of a 'practical' guide.

How Important is this Lesson?

This lesson is somewhat more advanced, and probably not essential to using bash at the start. It can also be hard to grasp, so if you're new to bash I recommend returning to it later if it's hard to understand the first time you read it.

However, knowledge of this area will catapult you to an elite of bash users that understand how terminals can be manipulated, and also enable you to understand how your prompt can be manipulated.

Non-Printable Characters

The terminal you use has what are described as *printable* characters and *non-printable* characters.

For example, typing a character like `a` (normally) results in the terminal adding an `a` to the screen. But there are other characters that tell the terminal to do different things that don't necessarily involve writing a character you'd recognise.

It's easy to forget this, but not everything that is sent to the computer is directly printed to the screen. The terminal *driver* takes what it is given (which is one or more bytes) and decides what to do with it. It might decide to print it (if it's a 'normal' character), or it might tell the computer to emit a beep, or it might change the colour of the terminal, or delete and go back a space, or it might send a message to the running program to tell it to exit.

When looking at non-printable characters, it's useful to be aware of a couple of utilities that help you understand what's going on. The first of these is a familiar one: `echo`.

Using `echo`

You're already familiar with echo, but it has some sometimes-useful flags you've not already used in this book:

```
echo -n 'JUST THESE CHARACTERS'
```

Type the above code into the terminal in this lesson.

Terminal

The `-n` flag tells echo not to print a newline.

```
echo -n $'JUST THESE CHARACTERS AND A NEWLINE\n'
```

Type the above code into the terminal in this lesson.

The `$` before the string makes us sure that bash will interpret `\n` as a newline on the screen, and won't just print out the characters `\n`. Try it without the `$` if you're unsure.

Here, it is the backslash character `\` that makes `echo` aware that a 'special' character is coming up. Being able to add a newline in this way means that you can send a newline to the terminal via the `echo` command, without confusing the command line by hitting the return key.

Other special characters include `\b` (for backspace), `\t` (for tab) and `\\` (to output a backslash):

Before you hit return, have a guess as to what this will output?

```
echo -n $'a\bb\bcde\bfg\b\b\b\n'
```

Type the above code into the terminal in this lesson.

If you guessed correctly, then well done! If you're struggling to understand what happened, note that a backspace does not delete the previous character, it just moves the cursor back a space.

You can also send a specific byte value to the terminal by specifying its hex value:

```
echo -n $'\x20\n'
```

Type the above code into the terminal in this lesson.

Think about that - you can use `echo` with these flags to control *exactly* what

gets sent to the screen. This is extremely valuable for debugging, or controlling what gets sent to the terminal.

It also bypasses the 'catching' of some characters you've seen from the previous readline lesson.

`CTRL-v` Escaping

Being able to output any binary value to the screen that we choose is useful, but what if we want to just output a 'special' character, and bypass the terminal's interpretation via terminal options or the readline library?

For example, if I hit `TAB` in my terminal it would normally *not* show a tab character (or move my cursor along a few spaces), as the readline library uses the `TAB` key (if hit twice in a row) to auto-complete any text we have not finished.

But if I'm typing something like:

```
echo 'I want a tab here:>X<a tab'
```

Type the above code into the terminal in this lesson.

How do I get a 'real' tab where the `X` is?

This is one way: instead of the `X`, you type `\C-v` and then `\C-i`.

If you type this in a terminal at the bash prompt, the cursor will tab across the screen in the way you might have previously expected.

If you look at the output of `stty -a` again (as you did in the previous readline lesson) then you will that `^V` is bound to the `lnext` action. I believe (but can't confirm) that this stands for 'next character literal'.

```
discard dsusp   eof     eol     eol2    erase   intr    kill    lnext
^O      ^Y      ^D      <undef> <undef> ^?      ^C      ^U      ^V
min     quit    reprint start   status  stop    susp    time    werase
1       ^\      ^R      ^Q      ^T      ^S      ^Z      0       ^W
```

There are multiple characters represented in this way. You've just seen tab (technically a *vertical tab*) represented as `^I`, and backspace represented as `^H`.

If you want to see a table of the possible shortcuts that may be seen, then type `man ascii` into the terminal above. The table begins like this:

```
020   16    10    DLE (data link escape)      120   80    50    P
-----------------------------------------------------------------------
000   0     00    NUL '\0' (null character)   100   64    40    @
001   1     01    SOH (start of heading)      101   65    41    A
002   2     02    STX (start of text)         102   66    42    B
003   3     03    ETX (end of text)           103   67    43    C
004   4     04    EOT (end of transmission)   104   68    44    D
005   5     05    ENQ (enquiry)               105   69    45    E
006   6     06    ACK (acknowledge)           106   70    46    F
007   7     07    BEL '\a' (bell)             107   71    47    G
010   8     08    BS  '\b' (backspace)        110   72    48    H
011   9     09    HT  '\t' (horizontal tab)   111   73    49    I
012   10    0A    LF  '\n' (new line)         112   74    4A    J
013   11    0B    VT  '\v' (vertical tab)     113   75    4B    K
014   12    0C    FF  '\f' (form feed)        114   76    4C    L
015   13    0D    CR  '\r' (carriage ret)     115   77    4D    M
```

How would you input this, therefore, and what will it output?

```
echo abcc^Hdefg
```

As the `echo` is processed, it should output:

```
abcdefg
```

because the second `c` is deleted by the `^H` you input by hitting `^V` (or `\C-v`) followed by `^H` (or `\C-h`).

Carriage Returns vs Line Feeds

The most commonly-seen non-printable character is the *carriage return*.

Carriage returns and line feeds cause much confusion, but it doesn't take long to understand the difference and (more importantly) why they are different.

If you think about an old-fashioned typewrite or printer that moves along punching out characters to a page, at some point it has to be told: 'go back to the beginning of the line'. Then, once at the beginning of the line, it has to be told: 'feed the paper up one line so I can start writing my new line'.

A *carriage return* is, as the word 'return' suggests, 'returns' the cursor to the start of the line. It's represented by the character `r` for return. The *line feed*, again as the name suggests, feeds the line up. In a modern terminal, this just means 'move the cursor down'.

So far, so clear and simple to learn. But, Linux does things differently! In Linux, `\n` is sufficient to do both. In Windows, you need both the `\r` and `\n` characters to represent a new line.

This means that files can 'look funny' in Linux terminals with these weird `^M` characters showing at the end of each line. To confuse things even more, some programs automatically handle the difference for you and hide it from you.

So what will this output?

```
echo $'Bad magazine\rMad'
```

The difficulty of understanding how bytes are turned into the output you see on the terminal is why it's important to have a way to see what the actual bytes in a file are. Here a very useful tool comes in: `hexdump`.

Hexdump

Run this:

```
echo $'Bad magazine\rMad' | hexdump
echo $'Bad magazine\rMad' | hexdump -c
```

Type the above code into the terminal in this lesson.

Hexdump prints out the characters received in standard input as hex digits. 16 characters are printed per line, and on the left is displayed the count (also in hex) of the number of bytes processed up to that line.

The `-c` flag prints out the contents as characters (including the control ones with appropriate backslashes in front, eg `\n`), whereas leaving it out just displays the hex values.

It's a great way to see what is *really* going on with text or any stream of output of bytes.

If you go back to the first example in this section:

```
echo 'JUST THESE CHARACTERS' | hexdump -c
echo -n 'JUST THESE CHARACTERS' | hexdump -c
```

Type the above code into the terminal in this lesson.

You can figure out for yourself the difference between using the `-n` flag in `echo` and not using it.

Terminal Escape Codes

A terminal escape code is a defined sequence of bytes that, if sent to the terminal, will perform a specific action.

Run this:

```
echo $'\033[?47h'
echo $'\033[?47l'
```

Type the above code into the terminal in this lesson.

The first line 'saves' the screen and the second restores it.

The ANSI codes always start with the `ESC` character (`033` in octal) and left bracket character: in hex `1B`, then `5b`. So you could rewrite the above as:

```
echo $'\x1b\x5b?47h'
echo $'\x1b\x5b?47l'
```

Type the above code into the terminal in this lesson.

These `ESC` and `[` characters are then followed by specific sequences which can change the colour of the screen, the background text, the text itself, set the screen width, or even re-map keyboard keys.

Type this out and see if you can figure out what it's doing as you go:

```
ansi-test() {
for a in 0 1 4 5 7
    do
    echo "a=$a "
    for (( f=0; f<=9; f++ ))
    do
```

```
        for (( b=0; b<=9; b++ ))
        do
            echo -ne "\\033[${a};3${f};4${b}m"

            echo -ne "\\\\\\\\033[${a};3${f};4${b}m"
            echo -ne "\\033[0m "
        done
        echo
    done
    echo
done
echo
}
```

Type the above code into the terminal in this lesson.

The output shows you what all the ANSI terminal escape codes are and you can see what they do in the terminal.

Sometimes when you `cat` a binary file, (or `/dev/random`, which outputs random bytes) the contents when output to a terminal can cause the terminal to appear to 'go haywire'. This is because these escape codes are accidentally triggered by the sequences of bytes that happen to exist in these files.

1

What will this output?

`echo -n $'12345\r54321'`

COMPLETED 0%

What You Learned

- What *terminal codes* are
- What *printable* and *non-printable* characters are
- How to output any arbitrary bytes
- How to input a literal character using `CTRL-v`
- The difference between `\n` and `\r\n`
- What *terminal escape codes* are

What Next?

Building on this knowledge, next you will learn how to set up your prompt so that it can show you (and even do) useful things.

Exercises

1) Research and `echo` all of `echo`'s escape sequences. Play with them and figure out what they do.

2) Research and `echo` ten terminal escape sequences.

3) Look up all the `CTRL-v` escape sequences and experiment with them.

4) Research the command `tput`, figure out what it does and rewrite some of the above commands using it.

5) Re-map your keyboard so it outputs the wrong characters using escape codes.

The Prompt

In this lesson, you will learn how to set up your main prompt, as well as other types of prompts and related shell variables.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- The `PS1` Variable
- The `PS2` Variable
- `PS3`
- `PS4`
- Pimp Your Prompt
- The `PROMPT_COMMAND` Variable
- What You Learned
- What Next?
- Exercises

Now that you've learned about escapes and special characters you are in a position to understand how the bash **prompt** can be set up and controlled.

How Important is this Lesson?

This lesson is not essential, but most people find it interesting and maybe fun to learn about.

The `PS1` Variable

Type this:

```
bash
PS1='My super prompt>>>>   '
ls
exit
```

Type the above code into the terminal in this lesson.

Terminal

As you'll remember, there are some *shell variables* that are set within bash that are used for various purposes. One of these is `PS1`, which is the prompt you see after each command is completed.

The `PS2` Variable

```
bash
PS2='I am in the middle of something!>>>  '
cat > /dev/null << END
some text
END
exit
```

Type the above code into the terminal in this lesson.

The PS2 variable is the 'other' prompt that the shell uses to indicate that you are being prompted for input to a program that is running. By default, this is set to `>` , which is why you see that as the prompt when you normally type the `cat` command above in.

`PS3`

`PS3` is used by the `select` looping structure. We don't cover that in this book as I've barely ever seen it used.

`PS4`

`PS4` is the last one:

```
bash
PS4='> Value of PWD is: $PWD'
set -x
pwd
cd /tmp
ls $(pwd)
cd -
exit
```

Type the above code into the terminal in this lesson.

In 'trace' mode `PS4` is echoed before each line of trace output. Do you

remember what trace mode is?

But why is the `>` in the output repeated? This indicates the *level* of indirection (ie subshells) in the trace. Every time the script goes one level 'down' a shell, the first character in the `PS4` value is repeated. Look at the output after the `ls $(pwd)` command.

> Note: Things can get really confusing if you have commands in your prompt, or you have `PROMPT_COMMAND` set (see below section). If you don't fully understand the output of the above, don't panic!

Pimp Your Prompt

For all the `PS` variables mentioned above, there are special escape values that can be used to make your prompt display interesting information.

See if you can figure out what is going on here:

```
bash
PS1='\u@\H:\w \# \$ '
ls
exit
```

Type the above code into the terminal in this lesson.

The table below may help you understand:

```
Escape value | Meaning                  | Notes
-------------------------------------------------------------------
\#           | Command number           | The number (starting
             |                          | from 1 and incrementing
             |                          | by one) of the command
             |                          | in this bash session.
-------------------------------------------------------------------
\$           | Root status              | If you have root, show a
             |                          | '#' sign, otherwise show
             |                          | '$'
-------------------------------------------------------------------
\t           | Current time             | In HH:MM:SS format -
             |                          | there are other formats
             |                          | possible with eg \T.
-------------------------------------------------------------------
```

```
\H            | Hostname                  | The hostname
              |                           | (fully-qualified)

---------------------------------------------------------------------
\w            | Current working           |
              | directory                 |
---------------------------------------------------------------------
\[            | Start control sequence    | Begin a sequence of
              |                           | non-printing characters,
              |                           | eg put a terminal
              |                           | control sequence in a
              |                           | prompt.
```

Use your knowledge gained so far to figure out what is going on here:

```
bash
PS1='\[\033[01;31m\]PRODUCTION\$ '
PS1='\[\033[01;32m\]DEV\$ '
exit
```

Type the above code into the terminal in this lesson.

Line 1 starts a fresh bash shell to work in so you don't 'taint' the existing shell with an overwritten prompt variable.

Line 2 is an example of setting the prompt to a control sequence that sets the colour of the prompt to red (`31m`) for a 'production' server.

Line 3 is an example of setting the prompt to a control sequence that sets the colour of the prompt to green (`32m`) for a 'production' server.

Line 4 exits the freshly-created bash shell and returns you to your original prompt.

You might want to try different numbers instead of `31` and `32` above to see what colours are set, and research which colours map to which numbers with an online search.

How would you make this automatically happen on a given server when you log in?

Show Hint

The `PROMPT_COMMAND` Variable

Another way the prompt can be affected is with the bash variable `PROMPT_COMMAND`:

```
bash
PROMPT_COMMAND='echo "Hello prompt $(date)"'
ls
exit
```

Type the above code into the terminal in this lesson.

Every time the prompt is displayed the `PROMPT_COMMAND` is treated as a command, and run.

You can use this for all sorts of neat tricks!

What You Learned

- What the `PS` variables are
- Where each `PS` variable is used
- How to augment your *prompts* to give you useful information
- How to automatically run commands before each *prompt* is shown

What Next?

Next you will learn a very useful technique for quickly creating files: the **here doc**.

Exercises

1) Look up the other prompt escape characters and use them.

2) Update your bash startup files so that the prompt tells you useful information when you log in

3) Create your own version of the `history` command by using the `PROMPT_COMMAND` variable.

Here Docs

In this lesson, we will cover 'here docs' and 'here strings', and when to use them.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Basic Here Docs
- More Advanced Here Docs
- Here Strings
- What You Learned
- What Next?
- Exercises

> Note: In this lesson, leading spaces are tabs. We have covered this in a previous lesson, but as a reminder: to get a tab character in your shell, type `\C-v`, and then hit the `TAB` button.

How Important is this Lesson?

Here documents, once learned, are used frequently in bash scripts or on the command line. It's a piece of knowledge that separates the experienced user from the junior.

Basic Here Docs

Type this in to see the basic form of the here doc:

The first line starts with cat followed by a redirection to a file.

Then you use two left chevrons and follow that with a string that represents a marker for the end of the file's contents. In this case, the word is `END`.

```
cat > afile.txt << END
```

Type the above code into the terminal in this lesson.

Terminal

Then you type in whatever you want the file to contain.

```
    A file can contain
            whatever you like
```

Type the above code into the terminal in this lesson.

(The whitespace in the second line above are spaces.)

When you're done, you can finish the file by typing the string you used as a marker on the first line alone on its own line:

```
END
```

Type the above code into the terminal in this lesson.

You can check the contents of the file by just `cat`ing it:

```
cat afile.txt
```

Type the above code into the terminal in this lesson.

The marker word does not need to be `END`! It could be anything you choose. `END` is generally used as a convention. Sometimes you see `EOF`, or `STOP`, or something similar. If you have a document with `END` in it, for example, you might want to avoid problems with the document ending early by choosing a different word.

More Advanced Here Docs

Now you're going to put a *here document* in a function. The function takes one argument. This argument is used as a filename, and the function creates a simple script with that filename that echoes the first argument given to that script.

Will this work? Read it carefully, predict the outcome, and then run it:

```
$ function write_echoer_file {
    cat > $1 << END
#!/bin/bash
echo $1
END
    chmod +x $1
}
write_echoer_file echoer
./echoer 'Hello world'
```

Type the above code into the terminal in this lesson.

- **Lines 1-7** creates a function called `write_echoer`
- **Lines 2-5** creates a file with the name of the first argument to the `write_echoer_file` function that `echo`es its own name
- **Line 6** makes the newly-created file executable
- **Line 8** uses `write_echoer_file` to create a program called `echoer`
- **Line 9** tries to call the newly-created `echoer` program to output `Hello world`

Hmmm. That didn't work, because the `$1` got interpreted in the write_echoer_file function as being the filename we passed in. In the *here doc*, we wanted the `$1` characters to be put into the script without being interpreted.

Try this instead:

```
function write_echoer_file {
    cat > $1 << 'END'
#!/bin/bash
echo $1
END
    chmod +x $1
}
write_echoer_file echoer
./echoer 'Hello world'
```

Type the above code into the terminal in this lesson.

Do you see the difference? This time, the *delimiter* word `END` was wrapped in single quotes. This made sure that the `echo $1` was not interpreted by the

shell when being typed in.

Can you see why we needed to use single quotes here? What happens when you use double quotes?

This kind of confusion can happen all the time when writing bash scripts, so it's really important to get these differences clear in your mind.

Our function is working now, but we could still make it better.

Try this (remember, the leading spaces are tabs - see the note above for how to input a tab):

```
function write_echoer_file {
    cat > $1 <<- 'END'
        #!/bin/bash
        echo $1
        END
    chmod +x $1
}
write_echoer_file echoer
./echoer
```

Type the above code into the terminal in this lesson.

The difference there was that we added a hyphen after the `<<` in **line 2**. This tells the shell to ignore leading tabs in the following text. This makes it much neater and easier to read scripts that use here docs, as the indentation can be consistent.

What if `END` is part of the *here doc* within another line?

```
function write_echoer_file {
    cat > $1 <<- 'END'
        #!/bin/bash
        echo $1
        echo Is this the END?
        END
    chmod +x $1
}
write_echoer_file echoer
./echoer
```

Type the above code into the terminal in this lesson.

No problem if it is not the only thing on the line.

Experiment to try and see what happens if it is.

Here Strings

Related to the *here doc*, a *here string* can be applied in the same way with the `<<<` operator:

```
function write_here_string_to_file {
    cat > $1 <<< $2
}
write_here_string_to_file afile.txt "Write this out"
```

Type the above code into the terminal in this lesson.

Here Docs Quiz

Q

What will this do?

```
HOME=asd
cat > afile << END
my home is $HOME
END
```

What You Learned

- What *here documents* are
- What *here strings* are
- How to create a *here document*
- How *here docs* and variables can be appropriately handled
- How to use *here docs* in a way that looks neat in a shell script

What Next?

Next we look at how bash maintains and uses a history of the commands run within it.

Exercises

1) Try passing a multi-line string to a here string. What happens?

Bash History

This lesson gives you a pragmatic overview of bash's history features, which can save you lots of time when at the terminal. You will learn about where your history is stored, how previous commands can be referenced, and various options and variables that can be set when working with your shell history.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Bash and History
- Using Your History
- How to Learn History Shortcuts
- More Advanced History Usage
- History Env Vars
- History Control
- `CTRL-r`
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Bash's history features are used at the command line so often that it's difficult to understate how important they are. It's a rich subject, but here I keep to the features I use most of the time.

Bash and History

Bash keeps a history of commands you have run. It keeps this in memory.

```
history
```

Type the above code into the terminal in this lesson.

Terminal

Using Your History

It can be tedious to type out often-used commands and arguments again and again, so bash offers several ways to save your effort.

Type this out and try and figure out what is going on:

```
echo nowhere
cd !$                              # !$ is replaced by 'nowhere', the last argument to the pr
echo 'About bash history' > file1
echo 'Another file' > file2
grep About file1
!!                                 # Repeat the last command
grep About file2
grep Another !$
rm file2
!e                                 # Repeat the last command beginning with 'e'
!gr                                # Repeat the last command beginning with 'gr'
```

Type the above code into the terminal in this lesson.

That introduced a few tricks you haven't necessarily seen before.

All of them start with the `!` (or so-called *bang*) sign, which is the sign used to indicate that the bash history is being referred to.

- The simplest, and most frequently seen is the double bang `!!`, which just means: re-run the previous command
- The one I use most often, though, is the second one you come across in the listing above: `!$`, or *bang dollar*. This one I must use dozens of times every day. It tells bash to re-use the last argument of the previous command.
- Finally, a *bang* followed by 'normal' characters re-runs the last command that matches those starting letters. The `!e` looks up the last command that ran starting with an `e` and runs that. Similarly, the `!gr` runs the last command that started with a `gr`, ie the `grep`.

Notice that the command that's rerun is the *evaluated* command. For that grep, what is re-run is as though you typed: `grep Another file2`, and not: `grep Another !$`.

How to Learn History Shortcuts

The history items above are enough to be going on with if you've not seen them before. There's little point listing them all as you'll likely forget them before you finish this course.

So before you go on, a quick note about learning these things: it's far more important to learn to *use* these tricks than *understand* them. To understand them is pretty easy - I'm sure you understood the passage above without much difficulty.

The way to learn these is to 'get them under your fingers' to the point where you don't even think about it. The way I recommend to do that is to concentrate on one of them at a time, and as you're working, remember to use that one where appropriate. Gradually you'll add more and more to your repertoire, and you will soon look like a whizz at the terminal.

More Advanced History Usage

You might want to stop there, as trying to memorise/learn much more in one go can be overwhelming.

But there are many more tricks to learn like this in bash, so I'm going to lay them out now so you might return to them later when you're ready.

Carrying on from where you left off above:

```
grep Abnother file1
^Ab^a^
```

Type the above code into the terminal in this lesson.

The carets (`^`) are used to replace a string from the previous command. In this case, `Ab` is replaced with: `a`. This is often handy if you made a spelling mistake.

Next up are the position command shortcuts, or 'word designators':

```
grep another file1 | wc -l
# Is that output correct? I want to check the file by eye:
cat !:2
```

Type the above code into the terminal in this lesson.

Starting with the *bang* sign to indicate we're referring to the history, there follows a colon. Then, you specify the word with a number. The numbers are zero-based, so the arguments start with `1`:

```
grep another file1
fgrep !:1-$          # A more sophisticated 'word designator'
```

Type the above code into the terminal in this lesson.

In the above example, you want to run the same command as before, but use the `fgrep` command instead of `grep` (`fgrep` is a 'faster' grep, which doesn't really help us here, but is just an example). To achieve this you use the so-called 'word designators'.

Here you add a dash indicating you want a 'range' of words, and the `$` sign indicates we want all the arguments up to the end of the previous command. Recall that `!$` means give me the last argument from the previous command, and so is itself a shortcut for `!:$`

Finally, another trick I use all the time:

```
LGTHWDIR=$(PWD)
cd /tmp
cat ${LGTHWDIR}/file1
cd !$:h
```

Type the above code into the terminal in this lesson.

The trick is the `:h` modifier to the `!$` history shortcut on **line 4**. This is one of several modifiers available, but the only one I regularly use.

When using a history shortcut, you can place a modifier at the end that starts with a colon. Here, the

- `!$` takes the last word from the previous command, (which you set to full directory path to the freshly-created `file1` file)
- Then, the modifier `:h` strips off the file at the end, leaving just the directory name. I use this all the time to quickly hop into a folder of a file I just looked at

History Env Vars

A quick note on environment variables that affect the history kept.

Type this in and try and figure out what's going on:

```
bash
HISTFILE=~/.bash_history
HISTTIMEFORMAT="%d/%m/%y %T "
history | tail
HISTTIMEFORMAT="%d/%m/%y "
history | tail
HISTSIZE=2
ls
pwd
history | tail
tail ~/.bash_history
exit
history
tail ~/.bash_history
```

Type the above code into the terminal in this lesson.

- The `HISTFILE` variable (**line 2**) sets the file that command history will be saved to when the shell exits
- The `HISTSIZE` variable (**line 7**) must be set to the number of commands you want stored in the `HISTFILE` file when bash exits
- There is also a `HISTFILESIZE` variable which determines the size of the history file itself. I did not get you to reduce the size of this to `1', as it would have wiped your history file and that would be more confusing! But you can play with it if you want
- Finally, the `HISTTIMEFORMAT` (**lines 3 and 5**) determines what time format should be shown with the bash history item. By default it's unset, so I usually set mine everywhere to be `%d/%m/%y %T`

You should have noticed that the `~/.bash_history` file did not get updated with the `ls` and `pwd` commands until bash exited. It's a common source of confusion that the bash history is not written out until you exit. If your terminal connection freezes, your history from that session may never be written out. This frequently annoys me!

History Control

There's another history-controlling environment variable worth understanding:

```
HISTCONTROL=ignoredups:ignorespace
ls
ls
 pwd    # <- note the space before the 'pwd'
pwd
ls
history | tail
```

Type the above code into the terminal in this lesson.

Was the output of history what you expected? `HISTCONTROL` can determine what gets stored in your history. The directives are separated by colons. Here we use `ignoredups` to tell history to ignore commands that are repeats of the last-recorded command. In the above input, the two consecutive `ls` commands are combined into one in the history. If you want to be really severe about your history, you can also use `erasedups`, which adds your latest command to the history, but then wipes all previous examples of the same command out of the history. What would this have done to the history output above?

`ignorespace` tells bash to not record commands that begin with a space, like the `pwd` in the listing above.

`CTRL-r`

Bash offers you another means to use your history.

Hit `CTRL` and hold it down. Then hit the 'r' key. You should see this on your terminal:

```
(reverse-i-search)`':
```

Let go. Now type `grep`. You should see a previous grep command. If you keep hitting `CTRL+r` you will cycle through all commands that had grep in them, most recent first.

If you want to cycle forward (if you hit `CTRL+r` too many times and go past the one you want (I do this a lot)), hit `CTRL+s`

one you want (I do this a lot)), hit `CTRL+S`.

History Quiz

1 What does this shortcut do?

`!$`

COMPLETED 0%

1 of 2

What You Learned

- Where bash keeps a *history* of commands
- How to refer to previous commands
- How to re-run a previous command with simple adjustments
- How to pick out specific arguments from the previous command
- How to control the *history* output
- How to control the commands that are added to the *history*
- How to search through your *history* dynamically

What Next?

Next you will tie these things together in a series of useful scripts that finish off this part.

Exercises

1) Remember to use one of the above practical tips every day until you don't think about using it. Then learn another one.

2) Read up on all the history shortcuts. Pick ones you think will be useful.

3) Amend your bash startup files to control history the way you want it.

4) Think about where your time goes at the command line (eg typing out directories or filenames) and research whether there is a way to speed it up.

Putting It Together - Some Useful Examples

In this lesson, you're going to see many of the ideas you've learned put together in more realistic contexts, so you can get a flavour for what bash can do for you on a day-to-day basis.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Output With Time
- Where Am I?
- `CTRL-r`
- Output Absolute File Path
- What Next?

How Important is this Lesson?

You can easily skip this lesson if you want as nothing here is new, but following this can embed some concepts and keep your motivation going before the final part!

Output With Time

Frequently, I want to get the output of a command along with the time. Let's say I'm running vmstat while a server is having problems, and I want to ensure that I've logged the time that vmstat relates to. Type this in:

```
function dateit() {
    while read line
    do
        echo "$line $(date '+ %m-%d-%Y %H:%M:%S')"
    done
}
vmstat 1 | dateit
```

Type the above code into the terminal in this lesson.

Terminal

> Note: `vmstat` is a program available on most Linux flavours that gives you a view of what the system resource usage looks like. It is not available on Mac OSes. If vmstat does not exist for you, then replace with `vm_stat`, which should be available.

You should be able to follow what's happening there based on what you've learned so far in

- Functions
- Loops
- Command Substitution
- Pipes and Redirects

The one exception is the `while read line` line. `read` is a shell builtin that takes input from standard input until a newline is seen (or an 'end of file' byte on its own line).

```
read
some input
```

Type the above code into the terminal in this lesson.

It returns a true exit code if input was seen:

```
echo $?
```

Type the above code into the terminal in this lesson.

and false if an end of file character is seen (with `\C-d`):

```
read
^D
echo $?
```

Type the above code into the terminal in this lesson.

If an argument is given to `read`, then a variable is populated with the input:

If an argument is given to `read`, then a variable is populated with the input.

```
read myinput
some input
echo $myinput
```

Type the above code into the terminal in this lesson.

So in the `dateit` function we input above, the while loop keeps taking input from standard input and acts on each line as it's read in, echoing each line to standard output until is gets an end-of-file character, at which point the while loop terminates as read returned a false exit code.

You will see the date appended to each line in the output. Experiment with the function to place the date before the line, or even on a separate line. See also the exercises below.

Where Am I?

You may be familiar with the `pwd` builtin, which gives you your current working directory (*cwd*). Similarly, there is an environment variable (`PWD`) that bash normally sets that stores the cwd.

```
pwd
echo $PWD
```

Type the above code into the terminal in this lesson.

Very often in scripts, you will want to know where the current working directory of the process is.

But also (very often) you will want to know where *the script you are running* is located *from within the script*.

For example, if you are running a script that is found in your `PATH` (ie not in your local folder), and you want to refer to another file relative to that script from within that script, then you will need to know where that script is located.

```
cat > /tmp/pwdscript.sh << 'EOF'
echo My pwd is: $PWD
echo I am running in: $(dirname ${BASH_SOURCE[0]})
EOF
```

```
EOF
chmod +x /tmp/pwdscript.sh
/tmp/pwdscript.sh
```

Type the above code into the terminal in this lesson.

Have a play with this command and read the code over to see what it does. If you can't work something out try the hint below.

Show Hint

Was the output of history what you expected? `HISTCONTROL` can determine what gets stored in your history. The directives are separated by colons. Here we use `ignoredups` to tell history to ignore commands that are repeats of the last-recorded command. In the above input, the two consecutive `ls` commands are combined into one in the history. If you want to be really severe about your history, you can also use `erasedups`, which adds your latest command to the history, but then wipes all previous examples of the same command out of the history. What would this have done to the history output above?

`ignorespace` tells bash to not record commands that begin with a space, like the `pwd` in the listing above.

`CTRL-r`

Bash offers you another means to use your history.

Hit `CTRL` and hold it down. Then hit the `r` key. You should see this on your terminal:

```
(reverse-i-search)`':
```

Let go. Now type `grep`. You should see a previous grep command. If you keep hitting `CTRL+r` you will cycle through all commands that had grep in them, most recent first.

If you want to cycle forward (if you hit `CTRL+r` too many times and go past the one you want (I do this a lot)), hit `CTRL+s`.

```
function extract() {
```

```
function extract() {
    if [ -z "$1" ]
    then
        echo "Usage: extract <file_name>.<zip|rar|bz2|gz|tar|tbz2|tgz|Z|7z|xz|ex|tar.bz2|tar.
    else
        if [ -f $1 ] ; then
           case $1 in
             *.7z)        7z x $1        ;;
             *.bz2)       bunzip2 $1     ;;
             *.exe)       cabextract $1  ;;
             *.gz)        gunzip $1      ;;
             *.tar.bz2)   tar xvjf $1    ;;
             *.tar.gz)    tar xvzf $1    ;;
             *.tar.xz)    tar xvJf $1    ;;
             *.tar)       tar xvf $1     ;;
             *.tbz2)      tar xvjf $1    ;;
             *.tgz)       tar xvzf $1    ;;
             *.Z)         uncompress $1  ;;
             *.xz)        unxz $1        ;;
             *.lzma)      unlzma $1      ;;
             *.rar)       unrar x -ad $1 ;;
             *.zip)       unzip $1       ;;
             *)           echo "extract: '$1' - unknown archive method" ;;
           esac
        else
            echo "$1 - file does not exist"
        fi
    fi
}
mkdir lbthw_misc
cd lbthw_misc
touch a b c
tar cvfz test.tgz a b c
rm a b c
extract test.tgz
```

Type the above code into the terminal in this lesson.

Challenge: explain what each line does.

You should be able to do this based on the lessons on:

- Tests
- Functions
- Loops

Output Absolute File Path

Quite often I want to give co-workers an absolute reference on a server to a file that I am looking at. One way to do this is to cut and paste the output of `pwd`, add a `/` to it, and then add the filename I want to share.

This takes a few seconds to achieve, and since it happens regularly, it's a great

candidate for a time-saving function:

```
function sharefiles() {
    for file in $(ls "$@"); do
        echo -n $(pwd)
        [[ $(pwd) != "/" ]] && echo -n /
        echo $file
    done
}
sharefiles
```

Type the above code into the terminal in this lesson.

You should be able to work out what is going on in the above function based on the lessons:

- Functions
- Loops
- Variables

The only novel element is the special parameter `$@`. This (very useful) variable gives you all the arguments passed to the function (or program, if called outside a function).

Saving time by writing a function is often a great idea, but deciding what is worth automating is a non-trivial task.

Here are some things you want to think about when deciding what to automate:

- How often do you perform the task?
- How much effort is it to automate (it's easy to under-estimate this!)
- Will the automation require effort to maintain?
- Do you always perform the task on the same machine?
- Do you control that machine?

My experience is that the effects of automation can be very powerful, but the above factors can also limit the return on investment.

Think about what you could automate today!

Think about what you could automate today?

What Next?

Well done! You've completed this part, and taken your bash knowledge from theoretical to practical. The next section - 'Advanced Bash' - covers the more sophisticated features of bash that will up your bash game to the next level.

Job Control

In this lesson, you'll learn what job control is, and how to manage and run multiple jobs in your shell.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Starting Jobs
- Controlling Jobs
- Waiting
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Job control is a core feature of bash, and considered a central concept to understand if you are using bash every day.

Starting Jobs

You're going to look at running a simple job using the `sleep` command.

Type this in:

```
sleep 60 &
```

Type the above code into the terminal in this lesson.

Terminal

You typed a 'normal' command (`sleep 60`) and then added in another character, the ampersand (`&`). The ampersand will run the command 'in the background', which becomes a *job* in this bash session.

The *job* has two identifiers, which are immediately reported to you in the terminal.

```
[1] 39165
```

The first is the *job number*, which in this case is `1`, and the second is the *process identifier*, in this case `39165`, but this will be different for you.

If, before the time is up, you run any other commands:

```
pwd
```

Type the above code into the terminal in this lesson.

then they are not interfered with by this running job. You can just carry on as normal.

If you wait for the program to finish (60 seconds in this case), and then run any other command, then bash will report to you what happened to that job. For example, if you wait, and type:

```
true
```

Type the above code into the terminal in this lesson.

You'll see:

```
[1]+  Done                    sleep 60
```

in the terminal.

Again, it reports the job number, this time with the status (`Done`), and the command that was originally run (`sleep 60`).

Controlling Jobs

Just like starting jobs, you can control jobs by sending signals to them.

Here you're going to start two jobs, one to sleep for two minutes, and the next

for one second more (so we can distinguish between them).

```
sleep 120 &
sleep 121 &
```

Type the above code into the terminal in this lesson.

Now you have two jobs running in the background. You can find out what their status is using the `jobs` builtin:

```
jobs
```

Type the above code into the terminal in this lesson.

and you should see this:

```
[1]-  Running                 sleep 120 &
[2]+  Running                 sleep 121 &
```

Each job is identified by its number in square brackets, followed by a plus or a minus sign. The plus sign indicates which job will be brought to the foreground if you run the `fg` builtin.

Let's do that here:

```
fg
```

Type the above code into the terminal in this lesson.

This time you should see:

```
sleep 121
```

The `sleep 121` process is now running in the foreground. Bash lets you know this by outputting the foregrounded command to the terminal.

Now send the `STOP` signal to the foregrounded process by hitting `\C-z`.

```
[2]+  Stopped                 sleep 121
```

Again, you see the job number, the plus sign, and the status (`Stopped`). If we

want the process to continue to run, then you need to use the `bg` builtin:

```
bg
```

Type the above code into the terminal in this lesson.

Now you should see:

```
[2]+ sleep 121 &
```

The process has been continued from its stopped state. If you've taken over two minutes to do all this then you may get a report that the process has terminated with a message that they are '`Done`'. If so, start the two sleep commands again and get back to here.

Now that you have the two sleeps running in the background, you can send other signals to them. For example, you can kill them:

```
kill %1
```

Type the above code into the terminal in this lesson.

```
[1]+  Terminated: 15          sleep 120
```

The per cent sign followed by a number is called a *job specification* and is the way you can tell bash you want to operate on that job number. Of course, you can also send a signal by using the process identifier as well.

Waiting

One technique that can be useful in bash scripts is to start a number of background processes, and then wait for them to finish before continuing. This can be done with the `wait` builtin.

If your original sleep is still running, then you can run wait.

```
wait
```

Type the above code into the terminal in this lesson.

Eventually, the wait command returns with a '`Done`' status reported for that

Eventually, the wait command returns with a `Done` status reported for that process.

```
[1]+  Done                    sleep 120
```

Normally, wait always returns an exit code of zero. But if you add a job specification, wait returns the exit code of the last-completed job.

```
sleep 20 &
sleep 30 && false &
wait %1 %2
```

Type the above code into the terminal in this lesson.

So what exit code will that `wait` report? Check with:

```
echo $?
```

Type the above code into the terminal in this lesson.

Challenge: What if you swapped the sleep time numbers 20 and 30 above?

1 One of the outputs when backgrounding a process is

COMPLETED 0%

1 of 2

What You Learned

- *Job control* using signals

- *Job specifications*

- The `fg` and `bg` builtins
- The `wait` builtin

What Next?

Next we look at *traps* and *signals*.

Exercises

1) Write a script to run a series of useful housekeeping or reporting commands in the background, and then output a summary message at the end when they have all completed.

Traps and Signals

In this lesson you will learn what a signal is, how the `kill` command can be used to send signals, and how signals can be trapped. In addition, you'll cover the 'wait' bash builtin, and what a 'process group' is.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Triggering signals
- The `kill` Command
- Trapping Signals
- Trap Exit
- A Note About Process Groups
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

Traps are an advanced concept. If you're new to bash you might want to follow this lesson to be aware of it, and apply it as you get more knowledge of Linux or go deeper into bash scripting.

Triggering signals

Any easy way to trigger a **signal** is one you will likely already have used.

Follow the instructions here:

```
sleep 999
```

Type the above code into the terminal in this lesson.

Terminal

Now hit the `CTRL` key down, hold it, and then hit the `c` key (`CTRL-c`). Then get the exit code:

```
echo $?
```

Type the above code into the terminal in this lesson.

You should have got the output of a number over 128. You will of course remember that `$?` is a special variable that gives the exit code of the last-run command.

What you are less likely to have remembered is that exit codes over 128 indicate that a signal triggered the exit, and that to determine the signal number you take 128 away from the number you saw.

Bonus points if you did remember!

Can you work out what the signal was that stopped the `sleep` command?

The signals are usually documented in the *signal* man page.

```
man signal
man 7 signal
```

Type the above code into the terminal in this lesson.

Show Hint

> Note: `man` pages have different sections. `man man` will explain more if you're interested, but to get a specific section, you put the section number in the middle, as above. Find out what section 7 is by reading `man man`. You might not have section 7 of the signal man page installed.

If the signals are not listed on the man pages on the lesson terminal, then google them!

Now figure out what the signal was, what the default action is for that signal

Now figure out what the signal was, what the default action is for that signal and the signal name that is triggered when you hit `CTRL-c`.

```
sleep 999
```

Type the above code into the terminal in this lesson.

Now hit the `CTRL` key down, hold it, and then hit the `z` key (`CTRL-z`). Then get the exit code:

```
echo $?
```

Type the above code into the terminal in this lesson.

Challenge: which signal does `CTRL-z` trigger?

The `kill` Command

Another way to send a signal to a process is another one you have also likely come across: the `kill` command.

The `kill` command is misnamed, because it needn't be used to terminate a process. By default, it sends the signal `15` (`TERM`), which (similar to `2`) usually has the effect of terminating the program, but as the name suggests, is a stronger signal to terminate than `INT` (*interrupt*).

```
sleep 999 &
KILLPID=$(echo ${!})
echo ${KILLPID}
kill -2 ${KILLPID}
echo ${?}
wait ${KILLPID}
echo ${?}
```

Type the above code into the terminal in this lesson.

> Note: The curly braces are required with the `${!}` (which surprised me!). Bash interprets the `!` as being a history command (try it!). I'm not sure why (it works fine outside the `$()`), but it is an indication that it's perhaps wise to get into the habit of putting curly braces around your variable names in bash.

Can you explain why the echo after the kill outputs `2` and not `130`?

Can you explain why the echo after the kill outputs `0` and not `130`?

Show Hint

Instead of `-2` in the above listing, you can use the signal name. Either `-INT` or `-SIGINT` will work. Try them.

Trapping Signals

Type out this first:

```
while :; do sleep 5; done
```

Type the above code into the terminal in this lesson.

Now hit `CTRL-c`. The while loop will stop. Now create a similar-looking file with an extra line:

```
cat > trap_exit.sh << END
#!/bin/bash
trap "echo trapped" INT
while :; do sleep 5; done
END
chmod +x trap_exit.sh
./trap_exit.sh # NOW HIT CTRL-c
```

Type the above code into the terminal in this lesson.

What's going on? In the second listing you used the `trap` builtin to inhibit the default response of the trap_exit process in the bash process and replace it with another response. In this case, the first argument to the `trap` builtin is evaluated and run as a command (`echo trapped`).

So how to get out of it and kill off the process?

First, hit `CTRL-z`, and then type:

```
kill %1
```

Type the above code into the terminal in this lesson.

Trap Exit

In addition to the normal signal name traps listed in the `man 7 signal` file, there are some special ones.

Type this out:

```
cat > trap_exit.sh << END
#!/bin/bash
trap "echo trapped" EXIT
sleep 999
END
chmod +x trap_exit.sh
./trap_exit.sh &
TRAP_EXIT_PID=${!}
kill -15 ${TRAP_EXIT_PID}
```

Type the above code into the terminal in this lesson.

- **Line 1-6** uses a here doc to create the `trap_exit.sh` script and then make it executable
- **Line 7** runs the script in the background
- **Line 8** uses the `${!}` variable to retrieve the backgrounded process identifier
- **Line 9** sends the `TERM` signal to the `trap_exit.sh` process

Which signal did we use there?

The `EXIT` trap catches the termination of the script and runs. Try it with `-2` as well.

Now run this:

```
./trap_exit.sh &
TRAP_EXIT_PID=${!}
kill -9 ${TRAP_EXIT_PID}
```

Type the above code into the terminal in this lesson.

Some of the signals are not trap-able! Why do you think this is?

Show Hint

Show Hint

Experiment with some other signals to determine how `EXIT` handles them.

What is the name of the `-9` signal? Is this the default that the `kill` command uses?

Show Hint

A Note About Process Groups

You may have noticed that in the above script you used the `wait` command after putting the process in the background.

The wait command is a bash builtin that returns when the child processes of the bash process completes.

This illustrates a subtle point about signals. They act on the *currently running* process, and not on their children.

Repeat the above exercise, but rather than having:

```
sleep 999 &
wait
```

type:

```
sleep 999
```

in your script.

What do you notice about the behaviour of the `EXIT` and `INT` signals?

How do you explain the fact that running this:

```
./trap_exit.sh
```

Type the above code into the terminal in this lesson.

and then hitting `CTRL-c` works to kill the sleep process and output 'trapped', where sending the signal 2 before did not?

where sending the signal -2 before did not?

The answer is that foregrounded processes are treated differently - they form part of a ‘process group’ that gets any signals received on the terminal.

If this seems complicated, just remember: `CTRL-c` kills all the processes ’going on’ in the foreground of the terminal with the `2` (or `INT`errupt) signal, while `kill` sends a message to a specific process, which may or may not be running at the time.

If this seems complicated, just remember: signals can get complicated!

Traps and Signals Quiz

1 What is `kill`'s default signal?

COMPLETED 0%

1 of 2

What You Learned

In this lesson you have learned:

- What a *signal* is
- What a *trap* is
- What the `kill` program does, and that it doesn't send `KILL` by default
- What an `INT` and `TERM` signal is
- How to *trap* exiting bash processes
- What a *process* group is, and its significance for *signals*

What Next?

Next we look at various methods used to debug bash scripts.

Exercises

1) Write a shell script that you can't escape from in the terminal provided above

2) Try and escape from the shell script you created in 1)

3) Ask everyone you know if they can escape the shell script

4) If no-one can escape it, send it to the author :)

5) Research the other 'special' signal traps. Use `man bash` for this.

Debugging

In this lesson, you will learn about bash flags useful for debugging, how to trace bash code, and useful resources for linting bash scripts.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Syntax Checking Options
- Controlling Jobs
- Managing Variables
- Profiling Bash Scripts
- Shellcheck
- What You Learned
- What Next?
- Exercises

How Important is this Lesson?

If you write, use or maintain bash of any complexity you'll want to know how to debug it!

Syntax Checking Options

Start by creating this simple (but broken) script:

```
cat > debug_script.sh << 'END'
#!/bin/bash
A=some value
echo "${A}
echo "${B}"
END
```

Type the above code into the terminal in this lesson.

Terminal

Now run it with the `-n` flag. This flag only parses the script, rather than running it. It's useful for detecting basic syntax errors.

```
bash -n debug_script.sh
```

Type the above code into the terminal in this lesson.

You can see it's broken. Fix it. Then run it:

```
bash debug_script.sh
```

Type the above code into the terminal in this lesson.

You'll see:

```
    [1]+  Done                    sleep 60
```

in the terminal.

Again, it reports the job number, this time with the status (`Done`), and the command that was originally run (`sleep 60`).

Controlling Jobs

Just like starting jobs, you can control jobs by sending signals to them.

Here you're going to start two jobs, one to sleep for two minutes, and the next for one second more (so we can distinguish between them).

```
bash -v debug_script.sh
```

Type the above code into the terminal in this lesson.

Try tracing to see more details about what's going on. Each statement gets a new line.

```
bash -x debug_script.sh
```

Type the above code into the terminal in this lesson.

Using these flags together can help debug scripts where there is an elementary error, or even just working out what's going on when a script runs. I used it only yesterday to figure out why a systemctl service wasn't running or logging.

Fix the error you see before continuing.

Managing Variables

Variables are a core part of most serious bash scripts (and even one-liners!), so managing them is another important way to reduce the possibility of your script breaking.

Change your script to add the 'set' line immediately after the first line and see what happens.

```
#!/bin/bash
set -o nounset
A="some value"
echo "${A}"
echo "${B}"
```

Now research what the `nounset` option does. Which `set` flag does this correspond to?

Show Hint

Without running this, try and figure out what this script will do. Will it run?

```
#!/bin/bash
set -o nounset
A="some value"
B=
echo "${A}"
echo "${B}"
```

Try it and see.

I always set `nounset` on my scripts as a habit. It can catch many problems before they become serious.

Profiling Bash Scripts

Returning to the `xtrace` (or `set -x`) flag, we can exploit its use of a PS variable to implement the profiling of a script:

```
#!/bin/bash
set -o nounset
set -o xtrace
declare A="some value"
PS4='$(date "+%s.%N => ")'
B=
echo "${A}"
A="another value"
echo "${A}"
echo "${B}"
ls
pwd
curl -q bbc.co.uk
true
```

Type the above code into the terminal in this lesson.

- **Lines 2-3** set the `nounset` and `xtrace` options.
- **Line 4** `declare`s a variable `A` and gives it a value.
- **Line 5** sets the `PS4` variable, which runs before each command is processed in bash, and outputs the result prepended by a `+` sign. From this point in the script, each line shows the time it was run to nanosecond granularity, allowing you to see where the time running the script goes.
- **Lines 6-14** runs various simple commands that exercise the ability to see how long each line runs for.

> Note: If you are on a Mac, then you might only get second-level granularity on the date!

Shellcheck

Finally, here is a very useful tip for understanding bash more deeply and improving any bash scripts you come across. Shellcheck is a website (http://www.shellcheck.net/) and a package that gives you advice to help fix

and improve your shell scripts. Very often, its advice has prompted me to research more deeply and understand bash better.

Here is some example output from a script I found on my laptop:

```
$ shellcheck shrinkpdf.sh
In shrinkpdf.sh line 44:
          -dColorImageResolution=$3             \
                                 ^-- SC2086: Double quote to prevent globb
ing and word splitting.

In shrinkpdf.sh line 46:
          -dGrayImageResolution=$3              \
                                ^-- SC2086: Double quote to prevent globbi
ng and word splitting.

In shrinkpdf.sh line 48:
          -dMonoImageResolution=$3              \
                                ^-- SC2086: Double quote to prevent globbi
ng and word splitting.

In shrinkpdf.sh line 57:
        if [ ! -f "$1" -o ! -f "$2" ]; then
                        ^-- SC2166: Prefer [ p ] || [ q ] as [ p -o q ] i
s not well defined.

In shrinkpdf.sh line 60:
        ISIZE="$(echo $(wc -c "$1") | cut -f1 -d\ )"
                      ^-- SC2046: Quote this to prevent word splitting.
                      ^-- SC2005: Useless echo? Instead of 'echo $(cm
d)', just use 'cmd'.

In shrinkpdf.sh line 61:
        OSIZE="$(echo $(wc -c "$2") | cut -f1 -d\ )"
                      ^-- SC2046: Quote this to prevent word splitting.
                      ^-- SC2005: Useless echo? Instead of 'echo $(cm
d)', just use 'cmd'.
```

The most common reminders are regarding potential quoting issues, but you can see other useful tips in the above output, such as preferred arguments to the `test` construct, and advice on 'useless' `echo`s.

What You Learned

In this lesson, you learned:

- bash flags useful for debugging

- How to use traps and `declare` to trace the use of variables
- How to make your scripts more robust with `nounset`
- How to use `shellcheck` to help you reduce the risk of your scripts failing

What Next?

Next you will look at *string manipulation* in bash.

Exercises

1) Find a large bash script on a social coding site such as GitHub, and run `shellcheck` over it. Contribute back any improvements you find.

String Manipulation

In this lesson, you'll cover how to edit strings in bash, what extglobs are and how you can use them, and how to avoid common quoting problems with strings.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- String Length
- String Editing
- String Editing
- And More...
- Extglobs and Removing Text
- Quoting Hell
- What You Learned
- What Next
- Exercises

Since so much of working in bash is related to files and strings of text, the ability to manipulate strings is valuable.

While tools such as `sed`, `awk`, `perl` (and many others) are well worth learning, in this lesson I want to show you what is possible in bash - and it may be more than you think!

How Important is this Lesson?

This lesson is not essential. String manipulation in bash is only occasionally useful to know about, for example when the above-mentioned tools (`sed`, `awk`, `perl` *et al*) are not available.

The quoting tricks in this lesson are occasionally very useful, however.

String Length

String Length

One of the most common requirements when working with strings is to determine length:

```
A='12345678901234567890'
echo "${#A}"                # How long is 'A'?
echo "$#A"
```

Type the above code into the terminal in this lesson.

Terminal

Why did the **line 3** not 'work'?

Show Hint

String Editing

Bash provides a way to extract a substring from a string. The following example explains how to parse *n* characters starting from a particular position.

Work out what's going on here. You may need to consult the manual:

```
echo ${A:2}
echo ${A:2:3}
```

Type the above code into the terminal in this lesson.

You can replace sections of scripts using search and replace. The first part enclosed in `/` signs represents what's searched for, and the second what is replaced:

```
echo "${A/234/432}"
echo "${A//234/432}"
```

Type the above code into the terminal in this lesson.

What's going on in the second command above? How does it differ from the

first?

String Editing

Another commonly-required string operation is getting a substring.

This outputs everything from the third character in the string:

```
echo ${A:2}
```

Type the above code into the terminal in this lesson.

and this outputs two numbers from the fourth character in the string:

```
echo ${A:3:2}
```

Type the above code into the terminal in this lesson.

You can tell from this that the strings are 0-indexed, that is the first character is numbered zero, the second is numbered one, and so on.

You can also search and replace within strings:

```
B=0000000000
echo ${B/0/1}
```

Type the above code into the terminal in this lesson.

The simple pattern replaces only the first `0` with a `1`. If you want the pattern to go across all the matches in the string, then add another slash:

```
echo ${B//0/1}
```

Type the above code into the terminal in this lesson.

You can also convert lower case to upper case and vice versa. Using a single comma will turn the starting title case character into a lower case character:

```
C=AbCd
echo ${C,}
```

Type the above code into the terminal in this lesson.

whereas using two will lower-case the entire string:

```
echo ${C,,}
C=aAbCd
```

Type the above code into the terminal in this lesson.

Similarly, the caret (`^`) character will upper-case a string also:

```
echo ${C^}
echo ${C^^}
```

Type the above code into the terminal in this lesson.

And More...

There are many more string manipulation tricks you can use, but it is not worth trying to learn them all in one go.

If you want to study them in more detail and find out what's available (or ever find yourself wondering what can be done in bash with strings) then have a look at the man page under *Parameter Expansion*.

Extglobs and Removing Text

A more advanced means of working with strings is possible by using bash's **extglobs** functionality.

A word of warning here: although this functionality is useful to know, it is arguably less useful than learning the programs `sed` and `perl` for this purpose. It's also quite confusing to have this extra type of glob syntax to learn in addition to regular expressions (and there's even flavours of those too!). Feel free to skip this section if you feel it's too obscure.

```
shopt -s extglob
A="12345678901234567890"
B="  ${A}  "
```

Type the above code into the terminal in this lesson.

You've ensured that the `extglob` option is on, and created a new variable `B`

You've ensured that the `extglob` option is on, and created a new variable `B`, which is `A` with two spaces in front and behind. The pipe (`|`) character is used in the `echo` output to show where the spaces are.

```
echo "B      |${B}|"
echo "B#+( ) |${B#+( )}|"
echo "B#?( ) |${B#?( )}|"
echo "B#*( ) |${B#\*( )}|"
echo "B##+( )|${B##+( )}|"
echo "B##*( )|${B##*( )}|"
echo "B##?( )|${B##?( )}|"
```

Type the above code into the terminal in this lesson.

Using the bash man page, and experimenting, can you figure out the differences between the above extglobs? They are:

- `?( )`
- `+( )`
- `*( )`

Now try `%` instead of `#` above. What happens?

One potentially handy application of this is when having to remove leading zeroes from dates:

```
TODAY=$(date +%j)
TODAY=${TODAY##+(0)}
```

Type the above code into the terminal in this lesson.

Remember: `#` is to the left, and `%` is to the right on a (US) keyboard. Or 'hash' is before 'per cent' in the alphabet.

Quoting Hell

Quoting - as I'm sure you've seen - can get very complicated in bash very quickly.

Try this:

```
echo 'I really want to echo $HOME and I can't avoid it'
```

Type the above code into the terminal in this lesson.

Uh-oh. You're now stuck.

Can you see why?

Hit `CTRL-c` to get out of it.

So you might think to try this:

```
echo "I really want to echo $HOME and I can't avoid it"
```

Type the above code into the terminal in this lesson.

but this time the `$HOME` variable is evaluated and the output is not what is wanted.

Try this:

```
echo 'I really want to echo $HOME and I can'"'"'t avoid it'
```

Type the above code into the terminal in this lesson.

That works. Remember this trick as it can save you a lot of time!

String Manipulation Quiz

1 What will this output?

```
echo ${#A}
```

What You Learned

This lesson taught you about string management using pure bash. This is a relatively rare field to cover, but worth having seen in case you come across it. You learned:

- How to edit strings using pure bash
- How to determine string length
- What *extglobs* are and how you can use them
- How to avoid common quoting problems

What Next

In the next lesson you will look at bash's **autocomplete** functionality.

Exercises

1) Learn how to do all of the above things in `sed` too. This will take some research and time.

2) Learn how to do all of the above in `perl`. This will also take some research and time!

3) Construct a useful `echo` that has a double-quoted string with a single-quoted string inside, eg one that outputs:

```
He said 'I thought she'd said "bash was easy when $s are involved" but tha
t can’t be true!'
```

Autocomplete

In this lesson, you will cover how autocomplete works in bash, the bash 'shift' builtin, and changing case in variables.

WE'LL COVER THE FOLLOWING

- How Important is this Lesson?
- Autocompleting Commands
- Autocompleting Arguments
- A Simple Program
- Adding Autocomplete Functionality
- Advanced Autocomplete Functionality
- Automate It
- What You Learned
- Exercises

How Important is this Lesson?

In this lesson you will build on what you learned previously about:

- Bash arrays
- Terminal escape codes
- Built-ins
- Bash functions
- Bash startup scripts

to comprehend more deeply a feature you will probably use almost every time you use bash.

Autocompleting Commands

If you hit the `TAB` key on your keyboard twice at the default prompt, then you should see something like this:

```
Display all 2266 possibilities (y or n)?
```

Terminal

If you're brave enough to hit `y` then you will see a list of all the commands available in your path.

To escape that list, hit `q`.

Now if you try typing `z` on a fresh shell command line, and then hit `TAB` twice, you will likely immediately see the commands available that begin with `z`.

That's autocomplete in bash - by default, you get the commands available that match the letters you've typed in so far. This is handy if you can only remember part of a command, or you just want to see what commands start with `a` (try it!).

Autocompleting Arguments

In addition to looking for commands, bash's autocomplete can be made to be context-aware. A simple example of this is to see what happens when you type `ls`, and then a space, and then `TAB` twice on a fresh shell line. What happens? You should see that this time commands are not listed, but something else...

Listing local files is something of a default in bash - it assumes that arguments are likely to be files. What I'm going to show you next is how you can get bash to be even more context dependent with auto-complete to the point where you can advise the user of your specific program about what is available to them.

A Simple Program

First you're going to create a program called `myecho`. As you type it in, try and work out what it's doing.

If you don't understand a line or command in there, then try and find out

what it does by playing with it in a toy script, or try and find out from `man bash`.

```
cat > myecho << 'END'
#!/usr/bin/env bash
function usage() {
        echo "Usage: $0 [red|green|blue] [message]"
        exit $1
}
if [ -z "$1" ]
then
        usage 1
fi
typeset -l COLOR="$1"
shift
RED='\033[0;31m'
GREEN='\033[0;32m'
BLUE='\033[0;34m'
MSG="$@"
COLOR=${COLOR,,}
if [[ "${COLOR}" = 'red' ]]
then
        MSG="${RED}${@}${RED}"
elif [[ "${COLOR}" = 'green' ]]
then
        MSG="${GREEN}${@}${GREEN}"
elif [[ "${COLOR}" = 'blue' ]]
then
        MSG="${BLUE}${@}${BLUE}"
else
        usage 1
fi
echo -e "${MSG}"
END
```

Type the above code into the terminal in this lesson.

If you're still struggling, then other lessons in this course will help enlighten you. The only new command here is `shift`. You may be able to work out what `shift` does from context, but just in case it's not obvious, it takes the first argument from your command line and removes it from the list of arguments passed in. This is handy for processing command line arguments, as you can keep calling `shift` until all the arguments are processed. Such a looping construct is used in many bash scripts.

You may also have not come across the variable `$@` before. Try finding out from the `man bash` page what it holds.

If you can't find it, don't worry, it's not easy to find in there. Look for the section on *Special Parameters*.

If you still can't figure it out, then try adding echo statements to figure out

what's going on. Keep at it.

Finally, can you figure out what the two commas in `${COLOR,,}` do? If not, play with it on the command line. This playing is how you learn and embed the bash knowledge as you get more advanced.

Now you've created the `myecho` script, make it executable and available on your path, and run it:

```
chmod +x myecho
PATH=.:${PATH}
myecho red WARNING this is dangerous
myecho green OK good to go
```

Type the above code into the terminal in this lesson.

Try changing the case of the colour argument (eg 'GreeN' and see what happens. Was that what you expected?). What part of the code is responsible for this?

Adding Autocomplete Functionality

Now let's say that it is some time later, and you've forgotten about your brilliant `myecho` script.

You type `myecho` on the command line, followed by a space and then hit `TAB` twice.

Wouldn't it be good if autocomplete could present the three colour options to you when auto-completing?

You can do that. Type out this script:

```
cat > myechocomplete << END
myecho_completion()
{
  COMPREPLY+=("red")
  COMPREPLY+=("blue")
  COMPREPLY+=("green")
}
complete -F myecho_completion myecho
END
```

Type the above code into the terminal in this lesson.

To make your shell aware of it, type:

```
source myechocomplete
```

Type the above code into the terminal in this lesson.

Now if you type `myecho`, followed by a space and then two 'Tab's, you should see the three options.

The `complete` command is a builtin, and the `-F` flag expects a function to be given to it, and will run it when the last argument to complete is typed on the command line and autocomplete is triggered.

When the function is run bash expects you to add to an array called `COMPREPLY` to give the options that should be made available to the user when they try to autocomplete. The three colours added above are the displayed to the user from the array.

Advanced Autocomplete Functionality

If you want to get a bit more sophisticated with bash's autocomplete, then you have other variables than `COMPREPLY` available to you. For this simple program you're going to provide a hint to input a message if the user has already input a colour argument to `myecho`.

```
cat > myechocomplete << END
myecho_completion()
{
  if [ ${COMP_CWORDS -ge 2 ]
  then
    COMPREPLY+=("Input")
    COMPREPLY+=("message!")
  else
    COMPREPLY+=("red")
    COMPREPLY+=("blue")
    COMPREPLY+=("green")
  fi
}
complete -F myecho_completion myecho
END
source myechocomplete
```

Type the above code into the terminal in this lesson.

Now if you type `myecho red`, followed by a space and then two 'Tab's, you should see the instruction on the line.

This is not exactly a neat way to get this information across. Unfortunately, the completion output functionality is not very flexible. You might want to try implementing a better solution with `echo`s or longer `COMPREPLY` entries to see its limitations.

Automate It

Although this is useful, it is not realistic to `source` the completion scripts every time you might want to use the particular command you have an autocomplete script for.

For this reason, you can add these functions to your bash startup scripts (see the lesson on Scripts and Startups in Part I) so that they are available whenever you use bash on that machine.

What You Learned

- How *autocomplete* works in bash
- How to create your own *autocomplete* function
- How to handle arguments to bash scripts and functions using `shift`
- How to change the case of string variables

Exercises

1) Write autocomplete scripts for already-existing programs you use often.

2) Look for autocomplete scripts on GitHub, and install and use them locally.

3) Read the bash manual section and figure out what all the `COMP_` variables store when autocompleting. Write example programs to use them.

Conclusion

In this conclusion we will wrap up the course and suggest where you go from here.

WE'LL COVER THE FOLLOWING

- Well Done!
- Where To Go Next
- Feedback

Well Done!

You've finished the course!

I hope you got the following benefits from this course:

- An overview of what bash is capable of
- An understanding of some of its key concepts, such as:
  - tests
  - exit codes
  - signals
  - options
  - subshells
  - readline
  - variables
  - command types
  - pipes and redirects
  - history
  - job control
  - autocomplete
  - traps

- An ability to read shell scripts

- Comprehension of - and the confidence to use - the jargon around bash

If I had to sum up what I hope this course gives you in one sentence it would be: confidence in reading the `man bash` pages without being confused or feeling lost

Where To Go Next

Although you've come a long way, your bash journey does not end here. I still learn new things about bash almost every day, and if you keep using it, you will too.

The best thing to keep your bash skills up and improve them is simply to use it. This should not be difficult, as the command line should now be your go-to place to getting everyday tasks done.

Here are some more suggestions for further work:

- Periodically read the `man bash` page
- Learn the most popular commands used on the command line
  - The GNU core utils are as good a place as any to start
- Write scripts for common daily tasks you or your colleagues do
- If you know C at all, or are curious, download the bash source code and scan through it for shell variables and the like that you have learned about

Feedback

If anything about this course did not work for you, or you have suggestions to improve, then please do get in touch with the author using the 'Send Feedback' button from any lesson.

If you particularly enjoyed a lesson, then please hit the 'Recommend' button so that others can be aware of it.